ADSDx: Towards Automated Accident Diagnosis for High-level Autonomous Driving Systems

ZONGAN HUANG*, Fudan University, China
JIARUN DAI*, Fudan University, China
MINGYUAN LUO, Fudan University, China
YUQI CHEN, ShanghaiTech University, China
HUIJIA SUN, ShanghaiTech University, China
YUAN ZHANG, Fudan University, China
MIN YANG†, Fudan University, China

Safety remains the primary obstacle to the widespread deployment of high-level autonomous driving systems (ADS). Therefore, manufacturers are in severe need of techniques to detect and mitigate safety-critical ADS faults. Although various ADS testing techniques have been proposed to identify accident-prone driving scenarios, reliable ADS-oriented accident diagnosis techniques remain scarce. This gap significantly hinders fault confirmation and mitigation. In light of this, we propose ADSDx, an automated tool dedicated to diagnosing safety-critical accidents of high-level ADSs. ADSDx is built on top of the high-fidelity ADS simulation platform. Given a reproducible accident scenario, ADSDx utilizes what-if reasoning to incrementally narrow down the candidate scope of accident-triggering conditions and accident-responsible ADS modules. In our evaluation, ADSDx demonstrated high effectiveness in diagnosing safety-critical accidents of industry-grade ADSs (e.g., Apollo and Autoware), achieving 95.0% precision and 100% recall in identifying accident-triggering driving conditions and 96.3% accuracy in locating faulty ADS modules. Furthermore, we demonstrated that ADSDx can help filter out 87.9% ~ 91.4% of fault-irrelevant messages, which is essential for investigating the in-depth root causes of ADS-involved accidents. Owing to this, we identified 10 code-level root causes of Apollo and Autoware, each of which can be stably exploited to cause collision accidents in diverse driving situations. To date, 6 of them have been confirmed by developers and 1 patch has been merged into the released version of ADS.

CCS Concepts: • **Software and its engineering → Software verification and validation**; • **Computer systems organization → Robotics**.

Additional Key Words and Phrases: autonomous driving system, accident diagnosis, simulation testing

*Both authors contributed equally to this research.
†Corresponding author

Authors' Contact Information: Zongan Huang, zahuang23@m.fudan.edu.cn, Fudan University, China; Jiarun Dai, jrdai14@fudan.edu.cn, Fudan University, China; Mingyuan Luo, myluo24@m.fudan.edu.cn, Fudan University, China; Yuqi Chen, chenyq@shanghaitech.edu.cn, ShanghaiTech University, China; Huijia Sun, sunhj2022@shanghaitech.edu.cn, ShanghaiTech University, China; Yuan Zhang, yuanxzhang@fudan.edu.cn, Fudan University, China; Min Yang, m_yang@fudan.edu.cn, Fudan University, China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2025 Copyright held by the owner/author(s).
ACM 1557-7392/2025/12-ART
https://doi.org/10.1145/3783993

Just Accepted

1 Introduction

Due to the potential to revolutionize the transportation system, autonomous driving systems (ADS) have received significant attention from investors. Currently, various companies [1, 8, 12] are actively developing high-level ADSs(see §2.1), and testing them on public roads. However, safety remains the main obstacle to the widespread deployment of high-level ADSs, considering the fact that ADS-involved accidents [5] continue to occur. In this state of confusion, manufacturers are in severe need of techniques to detect and mitigate safety-critical ADS faults (i.e., system flaws that might cause unsafe driving behaviors of the autonomous vehicle, see §2.3). To date, various ADS-oriented testing techniques have been proposed, e.g., scenario-based simulation testing [21, 34, 36, 41, 51, 52, 61, 68]. These works mainly aim to identify harsh driving scenarios (e.g., a surrounding vehicle is overtaking at a high speed), in which the autonomous vehicle might get involved in accidents.

However, few research efforts have elaborated on ADS-oriented accident diagnosis to understand how and why these accidents occur, largely hindering fault confirmation and mitigation. ADS-oriented accident diagnosis is expected to precisely determine contributing factors, encompassing both external triggers in the scenario and internal system anomalies. Most existing works [25, 35, 36, 52] in ADS testing manually diagnose the identified ADS-involved accidents in their publications, and pinpoint the faulty ADS module that is responsible for the accident, heavily relying on human expertise and experimentation. Some recent works [27, 46, 55] take a step further, establishing the capability of automated ADS accident diagnosis in simulated environments. Nonetheless, as thoroughly discussed in §2.3, these techniques either lack sufficient diagnostic capabilities (e.g., unable to infer external triggers or internal system anomalies) or rely on probability-based inference methods (e.g., inferring faults based on abrupt message changes in modules), resulting in the reporting of numerous irrelevant suspicious events or inaccurate diagnostic results. Without reliable and deterministic causality analysis, developers can hardly confirm and mitigate the underlying ADS faults behind these accidents.

Challenges. In light of this, we are highly motivated to develop a more deterministic ADS-oriented accident diagnosis methodology. With this methodology, we can effectively narrow down the scope of accident-relevant factors. To achieve this, we need to establish the causal relationships between each scenario attribute (i.e., the accident-triggering conditions) and the accident, as well as accurately identify the fault-inducing module (i.e., the accident-responsible module). However, it is obviously a challenging task to understand the causality of an ADS-involved accident, considering its complex nature:

- *Hard to Infer Accident-triggering Conditions.* As defined in §4.1, a scenario encompasses a variety of attributes, including environmental conditions (e.g., weather conditions) and traffic actors (e.g., vehicles and pedestrians with dynamic behaviors) surrounding the autonomous vehicle. The states of these attributes evolve continuously throughout the scenario. In theory, any of these attributes could influence ADS driving behaviors, making it hard to pinpoint the accident-triggering ones explicitly. For instance, in rear-end collision scenarios, accidents may result from factors including adverse weather conditions affecting perception, reduced friction on wet roads, and sudden lane changes by the leading vehicle. Moreover, it is difficult to determine the precise moments when these factors contribute to ADS misbehavior. This complexity arises because different scenario attributes can affect various ADS modules. Explicitly analyzing the causality between each attribute and the suspicious module outputs requires substantial expert knowledge and manual effort.
- *Hard to Localize Accident-responsible Module.* As summarized in [31], safety-critical ADS faults mainly derive from logical flaws (e.g., incorrect algorithm implementation). To diagnose logical flaws of conventional software, common practices [43, 54] suggest that code components, where the runtime state deviates from the desired one, should be the faulty ones. However, due to the complex functionality of high-level ADS, state deviations (e.g., 0.05m distance-measuring error in object detection) are everywhere, inevitable, and not necessarily responsible for an accident. Furthermore, in the pipeline-based modular

Just Accepted

architecture of ADS (as described in §2.1), upstream errors propagate downstream, causing output deviations in subsequent modules. This propagation makes it challenging to pinpoint which specific state deviation is the primary cause of an accident. Additionally, the desired runtime states of certain modules (e.g., the desired trajectory of the planning module) can hardly be determined.

Our Work. To address the above challenges, we propose ADSDx, an automated diagnosis tool for safety-critical accidents of high-level ADSs. ADSDx is built on top of the high-fidelity ADS simulation platform(detailed in §2.2), mainly because simulation testing offers sufficient flexibility in accident reproduction and ADS system monitoring. Given an ADS-involved accident that can be reproduced in the simulation platform, our key idea is to leverage what-if reasoning [17, 17, 22, 29, 33, 38], a fundamental scheme in causality analysis, to diagnose the accident in a more deterministic way. In general, it works by modifying each factor of system input (e.g., scenario attributes), or injecting new states into each system module, then re-running the system and re-monitoring the occurrence of the target accident. Through repeated trials, we can faithfully determine the causality between each system input/module and the accident. More specifically, given the disparate nature of external triggers (i.e., accident-triggering attributes) and internal system anomalies (i.e., the faulty ADS modules), ADSDx employs what-if reasoning to deliver two fundamental diagnostic capabilities, which are designed as follows.

- *Accident-triggering Condition Inference.* We aim to reduce the scope of accident-triggering conditions by trimming scenario attributes and time frames that are irrelevant to the accident. For scenario attribute trimming, we speculatively remove or modify each scenario attribute (such as traffic actors). For scenario time trimming, we remove scenario attributes sequentially over a period of time frames. Each time we will trim one factor (e.g., one scenario attribute or a period of time frames), replay the new scenario, and re-monitor the occurrence of the accident. If the accident still occurs after removing or modifying a factor, it indicates that the factor is irrelevant to the accident. As such, we can incrementally narrow down the space of possible accident-relevant factors.
- *Accident-responsible Module Localization.* We formalize the runtime states of the ADS as a series of runtime outputs of all ADS modules. Typically, these module outputs can be classified into two types: the environmental detection messages and the driving-decision messages. The environmental detection messages include outputs from the perception and prediction modules, which describe the status (e.g., position and size) of obstacles in the driving environment and their future movements. The driving-decision messages include outputs from the planning and control modules, which represent information about the runtime planned trajectory and vehicle control. Before diagnosis, we aim to construct the perfectly accurate messages for each ADS module output in the given accident scenario. For environmental detection messages, we can obtain absolutely accurate environmental information (e.g., the true position of obstacles) from the simulation environment. For driving-decision messages, we use the primary input to these modules (e.g., the current planned trajectory) as a reference for the ideal module outputs (e.g., controlling the vehicle to follow the planned trajectory). Next, we diagnose the ADS modules by injecting these perfectly accurate messages into each module during runtime. If even a perfectly accurate module cannot help avoid the accident, this module should not be responsible for the accident. As such, through one-by-one verification of the ADS modules, we can finally locate the compromised module. In particular, we design a customized diagnosis methodology for the planning module and customize the diagnostic order to ensure the effectiveness of our diagnosis.

Although such a what-if reasoning procedure sounds straightforward, as elaborated in §3 and §4, we address critical technical issues to ensure the usability and reliability of ADSDx when targeting complex high-level ADSs.

Results. We conducted a series of experiments to demonstrate the effectiveness, efficiency, and utility of ADSDx. Here, we chose two industry-grade ADSs as our evaluation targets (i.e., Apollo [1] and Autoware [6]), which have emerged as front-runners in the open-source community and have been widely adopted for commercial usage.

Just Accepted

Following existing practices of ACAV [55], we intentionally injected 22 faults into the stable versions of target ADSs to construct a benchmark dataset of 81 accident scenarios. Experimentally, ADSDx shows high effectiveness in this benchmark dataset, having 95.0% precision/100% recall in identifying accident-triggering scenario attributes, and 96.3% accuracy in determining the accident-responsible ADS modules. Compared to existing works (i.e., ACAV [55], ROCAS [27],DVCA [60]), ADSDx improves the accuracy of fault module identification by 26.3%~55.1%. Experiments also show that ADSDx's average diagnostic time is within 6 minutes, which is acceptable for an offline diagnostic tool.

To further demonstrate the utility of our work, we used ADSDx to assist the essential downstream task of the ADS safety assessment, namely fault-relevant message reduction, which is a fundamental step of in-depth root cause analysis. Statistically, ADSDx filtered out 87.9% ~ 91.4% of fault-irrelevant messages, reducing the size of input requiring analysis to a manageable level (i.e., 1.7~4MB). Owing to this, we successfully identified 10 code-level root causes of Apollo and Autoware. So far, 6/10 root causes have been confirmed. We also responsibly developed and submitted 5/6 patches to mitigate these safety-critical bugs, and till now, 1/5 patch has been merged into the released version of ADS. Lastly, our empirical case studies confirm that ADSDx generalizes effectively to the diagnosis of disparate ADS accident types, incurring only modest technical overhead.

Contributions. To sum up, this paper makes the following contributions.

- A mechanism of accident-triggering conditions inference for filtering accident-irrelevant scenario attributes and time frames effectively.
- A mechanism to localize the accident-responsible ADS module leveraging meticulous state modeling and injection.
- ADSDx, an automated diagnostic framework that can effectively identify the accident-triggering conditions and the accident-responsible module through what-if reasoning.
- We develop prototypes (see artifact [14]) to support the diagnosis of two high-level ADSs: Apollo [1] and Autoware [6].

2 Background

2.1 Autonomous Driving System (ADS)

2.1.1 Modular Architecture of High-level ADSs. As highlighted in a recent interview [45] with ADS industry players, high-level ADSs typically employ the modular architecture (see Figure 1(B)) rather than the end-to-end driving model. In this architecture, the driving pipeline is divided into modular tasks in sequential order. These modules interact with each other through the underlying runtime frameworks (e.g., Cyber-RT [2], ROS [11]) to achieve autonomous driving in real-time. ❶ **Perception** module receives sensor data (e.g., camera, LiDAR, etc.) for the detection and recognition of the external environment (e.g., obstacle detection); ❷ **Prediction** module is responsible for predicting future motions of surrounding obstacles, technically based on their historical behaviors; ❸ **Planning** module has the capability of decision-making and trajectory planning (i.e., to plan a safe path to the destination), based on the information obtained from the perception/prediction module; ❹ **Control** module accordingly generates concrete vehicle control commands (e.g., brake, throttle, etc.) to follow the planned trajectory.

2.2 ADS Simulation Platform

2.2.1 ADS Simulation Testing. By simulating various road conditions, weather conditions, and traffic situations, one can flexibly test the ADSs in diverse circumstances. In addition, simulation testing provides a controlled environment where repeatable tests can be performed without the risks of property damage or human injury. More importantly, the simulation platform enables real-time monitoring of the system states of the target ADS

Just Accepted

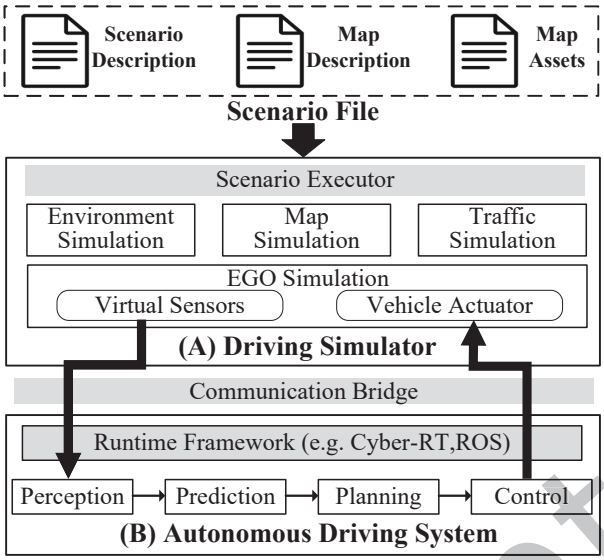


Fig. 1. Architecture of ADS Simulation Testing.

Just Accepted

and the environmental conditions, which are quite helpful for either accident detection or diagnosis. Considering the above, we decide to build ADSDx on top of the simulation platform. Specifically, given an ADS-involved accident that can be reproduced in the simulation platform, ADSDx faithfully pinpoints the accident-triggering conditions and accident-responsible modules.

2.2.2 Architecture of ADS Simulation Platform. As illustrated in Figure 1, the ADS simulation platform consists of two key components, namely the driving simulator and the communication bridge. ❶ **Driving Simulator** takes a scenario file as input to run simulation tests. The scenario file encodes configurations about the driving scenario in a machine-readable format (e.g., XML or JSON) with DSLs [4]. They quantitatively specify naturalistic settings (e.g., weather conditions), virtual maps (e.g., road networks and infrastructures), traffic conditions (e.g., motions of surrounding NPC vehicles and pedestrians), and `EGO` vehicle (e.g., vehicle dynamic model and virtual vehicle-equipped sensors). After parsing the scenario file, the simulator would utilize the graphics rendering engine to accordingly build the virtual world. ❷ **Communication Bridge** is responsible for the data exchange between the simulator and the target ADS. During simulation testing, the bridge would send virtual sensor data to the target ADS as inputs of the perception module, which are technically collected by the EGO-equipped virtual sensors in a simulated environment. After the computation of the ADS, the control module generates vehicle control commands, which are sent back to the simulator through the communication bridge to control the `EGO` movement. Notably, the implementation of such information exchange should be compatible with the ADS runtime framework (e.g., ROS2 [11] and Cyber-RT [2]), which manages the ADS-related communication tasks (e.g., between ADS and external devices or between ADS modules).

2.3 Motivation Example

2.3.1 An ADS-involved Accident Caused by Prediction Fault. Figure 2 illustrates an ADS-involved vehicle-to-vehicle collision caused by a safety-critical bug [3] in the prediction module of Apollo. In this scenario, the speed of the `NPC-E` vehicle is constantly changing, which makes the `EGO` unable to accurately predict its trajectory,

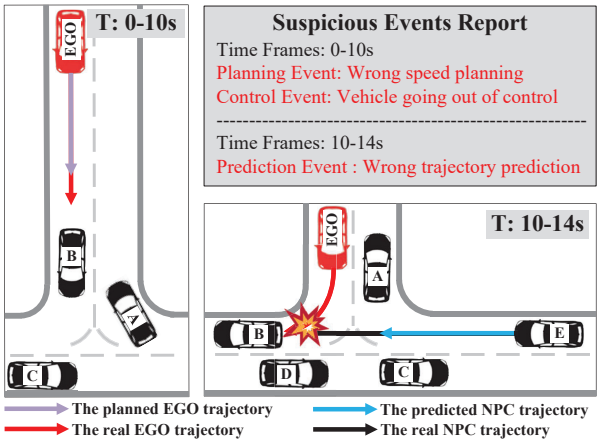


Fig. 2. Example of ADS-involved Collision. The EGO made a right turn near the intersection and failed to decelerate adequately, causing a head-on collision with NPC-E.

resulting in the collision. The underlying root cause of this accident stems from an unrobustness AI model integrated in the prediction module, which can not accurately perform the prediction task in complex intersection scenarios, causing a head-on collision between the autonomous vehicle (i.e., EGO) and a surrounding vehicle (i.e., NPC-E). To diagnose such an accident,a primary focus is on identifying the attributes of the scenario that directly contributed to the incident and pinpointing the critical moments within the scenario that are relevant to it. This approach helps narrow down the system inputs required to understand the root cause of the accident. For example, in the scenario depicted in Figure 2, we need only trace the ADS code handling inputs related to NPC-E within a specific time window, rather than analyzing every moment of data processing related to each NPC. Furthermore, as ADS comprises multiple complex modules, the diagnostic process for failures in different modules necessitates distinct areas of expertise. Therefore, it is crucial to identify which specific ADS module failure is responsible for the accident, enabling us to allocate further fault analysis to developers with the relevant expertise. For instance, diagnosing failures in the control module in the scenario shown in Figure 2 requires a comprehensive understanding of vehicle dynamics and control algorithms, whereas diagnosing prediction module failures often requires expertise in deep learning models in trajectory prediction.

2.3.2 Existing Practices in Diagnosing ADS-involved Accidents. Currently, some studies [16, 27, 46, 55, 60] have attempted to automate accident diagnosis to identify accident-relevant factors (e.g., scenario attributes and faulty ADS modules). DiaVio [46] automates the identification of responsible parties in accidents (i.e., the primary at-fault vehicle) by fine-tuning large language models on extensive accident reports. However, it cannot diagnose specific ADS module failures. ROCAS [27] and ACAV [55], take a step further, establishing the capability of automated ADS faulty module diagnosis in simulated environments. ROCAS [27] identifies the faulty module by comparing message variations between two similar scenarios (one resulting in an accident and one that does not), tagging the first module with abrupt message changes as faulty. This approach, however, is unreliable and risks misdiagnosis as minor scenario modifications may substantially impact different ADS modules. For example, adjusting the speed of NPC-E in Figure 2 can prevent the accident. Such modifications alter the outputs of perception and prediction modules simultaneously, potentially leading to incorrect fault attribution to the perception module. ACAV [55], on the other hand, is to identify a wide range of suspicious events observed during the scenario execution (e.g., critical vehicle interactions), and correlate them with internal system messages

Just Accepted

(e.g., runtime messages produced by ADS modules) to locate the potential faulty modules. However, without reliable and deterministic causality analysis, ACAV can hardly determine whether these suspicious scenario events and potential faulty modules are exactly accident-responsible ones, resulting in many inaccurate diagnosis results. For instance, in Figure 2, these suspicious events (i.e., unreasonable speed planning and inaccurate vehicle control) reported by ACAV during the 0–10s timeframe did not lead to a collision as `NPC-B` accelerated away from the intersection later. Unlike previous approaches, some studies [16, 60] adopt a modify–replay–observe paradigm to isolate irrelevant factors in accidents, a well-established high-level idea for fault diagnosis. As discussed in §9.2, this strategy is common not only in ADS-oriented diagnosis but also across other domains, such as network protocols [28, 48, 49] and cyber-physical systems [22]. Although these methods share conceptual similarities with our "what-if" reasoning, their problem modeling for ADS accident diagnosis suffers from notable limitations in both comprehensiveness and rigor. Specifically, DVCA [60] models the problem solely from the system perspective and lacks reliable diagnostics for assessing the responsibility of the planning module (i.e., it does not determine whether the scenario provides feasible planning space). Its core idea is that if a driving violation disappears after removing certain system-side components or messages, those removed elements are likely the root cause of the violation. However, this approach neglects the influence of scenario-side factors on diagnostic outcomes. For example, altering system-side messages may disrupt the necessary accident-triggering conditions on the scenario side, resulting in unreliable diagnoses. Less_Is_More [16], on the other hand, focuses exclusively on the scenario perspective, which constrains its capacity to identify faulty modules within ADS. These limitations not only impede the effective elimination of accident-irrelevant factors but also undermine diagnostic accuracy, as demonstrated in §5.

In contrast, ADSDx models ADS accident diagnosis from both scenario and system perspectives, offering more rigorous and deterministic diagnostic oracles.

3 Overview

3.1 Overall Idea

What-if reasoning [17, 22, 29, 33, 38] is a fundamental scheme in causality analysis. Specifically, it can decompose the complex diagnosis task into multiple manageable sub-tasks, each of which independently investigates the correlation between the fault occurrence and an individual system component/input. In such a manner, one can confidently narrow the scope of potential root causes. Inspired by this, we are motivated to explore the possibility of applying what-if reasoning in diagnosing ADS-involved accidents. ❶ For identifying fault-triggering conditions, we pose the hypothetical question: *"What if a specific attribute of the scenario is eliminated or changed, would the accident still occur?"*. To answer this, we eliminate or change each possible attribute of the scenario, and then re-run the scenario in the simulation platform to monitor the occurrence of the target accident. If the accident still occurs, the eliminated or changed attribute should not be responsible for the accident. ❷ For identifying faulty code modules, our hypothetical question is: *"What if a specific ADS module works correctly, would the accident still occur?"*. To test this hypothesis, we try to inject ground-truth states into an ADS module each time, and then re-run the driving scenario. If the accident still occurs, the injected module should not be responsible for the accident, at least not primarily.

3.2 Design Considerations

In this subsection, we respectively introduce our design considerations for identifying the fault-triggering conditions and faulty modules based on the ADS-specific what-if reasoning.

3.2.1 Accident-triggering Condition Inference. Following the idea introduced in §3.1, an intuitive solution is to modify only one scenario attribute while keeping the others unchanged during each test. However, in practice,

Just Accepted

there might exist some scenario attributes that have runtime interactions with the others. For instance, removing an accident-irrelevant NPC vehicle might cause a following accident-relevant NPC vehicle to accelerate, thereby affecting the occurrence of the accident. To address this, ADSDx records the exact runtime behavior (i.e., the exact trajectory) of each NPC vehicle, and then curates a semantically equivalent scenario configuration file in which each NPC has stable motions without any non-deterministic interactive runtime behaviors.

Meanwhile, from the perspective of efficiency, it is not a good choice to eliminate accident-irrelevant NPC traffic actors one-by-one, particularly in congested scenarios with numerous NPCs. Here, ADSDx introduces a scenario trimming algorithm to boost the inference procedure, efficiently eliminating a substantial portion of definitely accident-irrelevant NPCs based on their spatiotemporal correlation with the `EGO` vehicle.

3.2.2 Accident-responsible Module Localization. To achieve state injection, we choose to disable the under-verified module and replace the runtime outputs of this module with the perfectly correct ones. If the accident still occurs, the under-verified module should not be the main cause of the accident. To ensure the correctness of state injection, as clarified in §4.3, we should first meticulously model the perfect states for different ADS modules (i.e., runtime messages with perfectly correct syntax and semantics at any timestamp). Then, we construct these states through data monitoring for the simulation environment and ADS internal communication. After that, we can inject them into the specified ADS module via the communication bridge and the ADS runtime framework (see §2.2).

Additionally, to avoid error propagation mentioned in §1, it might be straightforward to merely diagnose the ADS modules one after another strictly following the modular pipeline (see Figure 1(B)). However, in practice, the modeling of the perfect state is not achievable for all ADS modules. In particular, the planning module is responsible for high-level decision-making that generates a safe trajectory to the destination. Obviously, there might exist numerous feasible trajectories, and it is hard to subjectively determine the perfectly correct one. For comparison, other modules are proposed for low-level decision-making (e.g., controlling the vehicle to follow the specified trajectory) or environmental perception and monitoring, whose ideal runtime states (i.e., ground-truth outputs) can be easily determined. In light of this, ADSDx designs a customized diagnosis methodology for the planning module, which works by verifying the presence of accident-free trajectories at each possible timestamp during the runtime simulation testing. Accordingly, we customize the order of the diagnostic pipeline to mitigate potential inaccuracies caused by this customized diagnosis methodology.

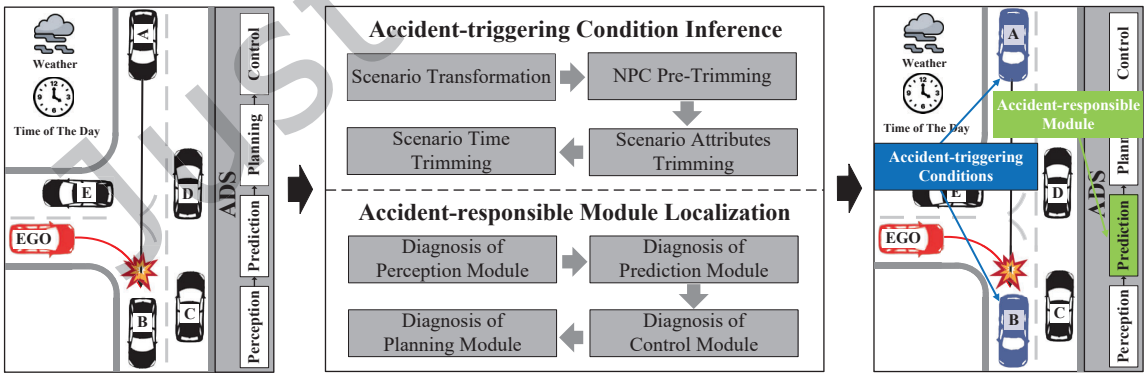


Fig. 3. Overview of ADSDx.

Just Accepted

3.3 Workflow

Integrating the aforementioned design considerations, Figure 3 illustrates the general workflow of ADSDx. ADSDx comprises two essential diagnostic capabilities, namely *Accident-triggering Condition Inference* and *Accident-responsible Module Localization*.

4 Approach

The structure of this section is organized as follows: §4.1 formally presents the diagnostic objectives and approach for the problem of ADS accident diagnosis, along with the relevant symbol definitions and formulas. §4.2 and §4.3 then provide detailed technical implementations for accident-triggering condition inference and accident-responsible module localization, respectively.

4.1 Formalization of ADS Accident Diagnosis

4.1.1 Scenario Definition. A driving scenario can be represented as a collection of scenes (denoted as $Scene_t$) [59].

$$\mathbb{S} = \{Scene_t | t \in [0, T]\}, \quad Scene_t = ENV_t \cup OBSTACLE_t \cup ROAD_t$$

Here, the T represents the duration of this scenario. Each scene captures a snapshot of all scenario attributes as well their status at a specific timestamp t. These scenario attributes can be classified as the following categories. ❶ **Environment Settings** (denoted as ENV) are defined to describe the natural environment (e.g., weather conditions, time-of-the-day, etc.). ❷ **Scenario Obstacles** (denoted as $OBSTACLE$) refer to the surrounding static obstacles or dynamic actors (e.g., pedestrians and vehicles) which can be also regarded as the NPCs in a driving scenario. Notably, many scenario obstacles do not have predefined runtime states across different timestamps; rather, their states are dynamically determined through runtime interactions with the EGO vehicle and other obstacles (e.g., NPC vehicles' behaviors respond to the EGO's driving actions). ❸ **Road Conditions** (denoted as $ROAD$) refer to the road structures (e.g. intersection, multi_lanes) involved in the scenario, where the EGO vehicle are driving on. During runtime, ADS would frequently interact with these scenario attributes to ensure safe driving.

4.1.2 ADS Runtime States. The ADS can be divided into multiple functional modules as introduced in §2.1. During each driving decision cycle, these modules generate the comprehensive runtime states (i.e., the module outputs) of the ADS (denoted as $\mathbb{R}$), which can be expressed as follows:

$$\mathbb{R} = \{State^{Perc}, State^{Pred}, State^{Plan}, State^{Cont}\}$$

Here the $State^X$ refers to the runtime states of each ADS module which will be formally represented in the following.

- **Perception States (marked as $State^{Perc}$).** The perception module is designed mainly for recognizing the surrounding objects of the EGO vehicle in a real-time fashion. Specifically, the semantics of the perception states include the status of the each obstacle perceived by ADS in the scenario (e.g., its position, rotation, velocity, etc).
- **Prediction States (marked as $State^{Pred}$).** The prediction module predicts the future trajectory of each surrounding object of the EGO vehicle. At each timestamp, the prediction module should return a list of prediction results, and each element of it represents the status of a perceived obstacle in the near future (detailed as follows).
- **Planning States (marked as $State^{Plan}$).** The planning module generates EGO's future trajectory based on the current driving task and environment conditions. At each timestamp, it outputs a trajectory starting from the current position of EGO vehicle. The planned trajectory, expressed as follows, comprises a series of GPS waypoints with fixed time intervals (e.g., 0.1s), encompassing position, velocity, etc.

Just Accepted

- **Control States (marked as $State^{Cont}$).** The control module produces vehicle control commands (i.e., brake, throttle, etc.) to make the EGO vehicle follow the planned trajectory. At each timestamp, the control module should return the value of each control command that needs to be applied for the EGO vehicle chassis.

4.1.3 ADS Scenario Execution. The outcomes of each scenario execution (denoted as $\mathbb{E}$) are jointly determined by the driving scenario (i.e., $\mathbb{S}$) and the ADS runtime states (i.e., $\mathbb{R}$).

$$\mathbb{E} = Execution(\mathbb{S}, \mathbb{R})$$

These outcomes can typically be represented as the states of traffic participants (e.g., the positions and rotations of EGO vehicle and scenario obstacles) at each timestamp (i.e., t).

4.1.4 ADS Accident. Upon detecting accidents in ADS scenario execution results, we employ distinct accident features to classify different accidents.

$$Accident(\mathbb{E}) = \begin{cases} \text{accident_features}, & \text{If accident occurs} \\ NONE, & \text{Otherwise} \end{cases}$$

Specifically, when detecting collision accidents, we use the four discriminative features including collision objects (i.e, the EGO vehicle and involved obstacles), collision angle (e.g., side-on, head-on, or rear-end collisions), the relative speeds of collision objects, and collision location (i.e., the lane position where the collision occurs).

4.1.5 Accident-triggering Condition Inference. For accident-triggering condition inference, our objective is to determine both the set of scenario attributes relevant to the accident and the relevant time frames of the scenario. Consequently, our diagnostic procedure consists of the following two scenario trimming steps:

- **Scenario Attributes Trimming($D_{\mathcal{A}}$).** We consider a scenario attribute x as accident-irrelevant if the modified scenario $S \setminus \{x\}$ (where x is removed from S's all scenes) can still trigger the original accident. Thus, the set of all accident-irrelevant attributes in S is given by:

$$Irre(\mathbb{S}) = \{x | x \in \mathbb{S}, \quad Accident(\mathbb{E}^{\mathbb{S}\setminus\{x\}}) == Accident(\mathbb{E})\}$$

  where $\mathbb{E}^{\mathbb{S}\setminus\{x\}}$ denotes the scenario execution results of the modified scenario $\mathbb{S} \setminus \{x\}$. Then, the scenario attributes trimming results in a new scenario $\mathbb{S}_{trim}$ without all the accident-irreverent scenario attributes.

$$D_{\mathcal{A}}(\mathbb{S}, \mathbb{R}) = \mathbb{S}_{trim} = \mathbb{S} \setminus Irre(\mathbb{S})$$

- **Scenario Time Trimming ($D_{\mathcal{T}}$).** Similar to the definition of accident-irrelevant scenario attributes, we consider scenario time frames (i.e., $t \notin [t1, t2]$) as accident-irrelevant if the trimming scenario $\mathbb{S}[t1, t2]$, still trigger the original accident.

$$\mathbb{S}[t1, t2] = \{Scene_t | t \in [t1, t2]\}$$

  Since the scenes occurring after the collision time (T_c) are inherently irrelevant to the accident, we transform scenario time trimming into identifying the maximum trimming moment $t_{\mathcal{T}}$, such that the trimming scenario $S[t_{\mathcal{T}}, T_c]$ still triggers the accident.

$$D_{\mathcal{T}}(\mathbb{S}, \mathbb{R}) = \mathbb{S}_{trim}[t_{\mathcal{T}}, T_c] = \underset{\mathbb{S}_{trim}[t, T_c]}{\operatorname{argmax}}\{t \in [0, T_c] | Accident(\mathbb{E}^{\mathbb{S}_{trim}[t, T_c]}) == Accident(\mathbb{E}^{\mathbb{S}_{trim}[0, T_c]})\}$$

In §4.2, we will provide technical details to explain how we ensure the effectiveness and efficiency of scenario trimming.

Just Accepted

4.1.6 Accident-responsible Module Localization. Typically, an ADS accident can be attributed to one or more faulty runtime states of the ADS modules. Here, we consider the runtime states of a module as accident-relevant faulty states if improving or correcting those states can help prevent the accident. To identify such faulty states, we further define the perfect module state as the runtime state that fully meets the functional expectations of the specified module in the given scenario, both syntactically and semantically. For modules whose perfect states are observable, we can diagnose whether the original module state is faulty via state injection. However, as discussed in §3.1, the perfect planning state is unable to obtain. Thus, we design a dedicated diagnostic approach for the planning module to determine whether an alternative planning output could have prevented the accident. Accordingly, our accident-responsible module localization can be formalized as follows:

$$\mathbb{A} = \{X | State^{X} \in R \setminus \{State^{Plan}\} \ \ \& \ \ Accident(\mathbb{E}^{\mathbb{S}_{trim}}_{P(\mathbb{R},X)}) == NONE\}$$

$$D_{\mathcal{M}}(\mathbb{S}, \mathbb{R}) = \begin{cases} \text{argmin}_{State^{X} \in R/\{State^{Plan}\}}\{X \in \mathbb{A}\}, & \mathbb{A} \neq \emptyset \\ State^{Plan}, & \mathbb{A} = \emptyset \ \ \& \ \ (\forall t \in (t_{\mathcal{T}}, T_c], \mathbb{DA}(\mathbb{S}, t_{\mathcal{T}}, t)! = \emptyset) \\ NONE, & Otherwise \end{cases}$$

Here, $P(\mathbb{R}, X)$ refer to the updated runtime states under the perfect module state injection for the module X and $\mathbb{E}^{\mathbb{S}_{trim}}_{P(\mathbb{R},X)}$ refers to the scenario execution result of the trimming scenario $\mathbb{S}_{trim}$ under these updated ADS runtime states. If all modules except the planning module are fault-free, we then assess the planning module itself. This assessment determines whether, during the critical pre-collision period (i.e., $(t_{\mathcal{T}}, T_c]$), feasible drivable areas ($\mathbb{DA}$) always existed for the `EGO` vehicle to avoid the accident. If such areas existed, the original planning output is deemed faulty; otherwise, the ADS bears no functional responsibilities. The technical details for the state injection and drivable area calculation that underpin this diagnostic approach are elaborated in §4.3.

4.2 Accident-triggering Condition Inference

4.2.1 Overview of Diagnosis Methodology. Following the problem definition in §4.1, we begin by trimming scenario attributes. ❶ (**Scenario Attributes Trimming**) To accomplish this task, we modify or remove one scenario attribute at each time, and re-run the simulation scenario to monitor the occurrence of the accident. If the accident still occurs, this attribute is not responsible for the accident. Specifically, environment settings are crucial configurations of a driving scenario, which cannot be simply eliminated. Hence, we assign non-harsh default value to each naturalistic setting for scenario replaying (e.g., set `time-of-the-day` to "10 a.m.", ensuring sufficient lighting for perception). For each scenario obstacle, we simply remove it during scenario replaying to verify whether it is relevant to the accident occurrence. Notably, the `EGO`'s driving decisions continuously rely on road conditions for path planning. Thus, we consider road conditions to be inherently relevant to the accident and do not subject them to trimming. ❷ (**Scenario Time Trimming**) After that, we employ a binary search algorithm to narrow down the time range that is truly relevant to the accident. Specifically, for each chosen time t (where $t < T_c$), we modify or remove the states of all scenario attributes during the time interval $[0, t)$ using the previously mentioned method, while keeping the scenario attributes unchanged from $[t, T_c]$. We then rerun the new scenario. If the accident occurs in the same manner as in the original scenario, the time frame $[0, t)$ is deemed irrelevant to the accident. During scenario replay, immediate collisions may occasionally occur due to the sudden appearance of vehicles. To avoid confusing these with the original collision, we check the accident features mentioned in §4.1 (e.g., collision location and involved objects).

Most importantly, we introduce both scenario transformation and NPC pre-trimming strategies to improve the effectiveness and efficiency of the diagnosis.

Just Accepted

4.2.2 Scenario Transformation. In practice, existing ADS testing techniques [34, 36, 40, 41, 58, 68] commonly rely on intelligent NPCs [7, 10] (i.e., a built-in capability of the driving simulator [26, 47]) to ease the construction of complex simulation scenarios. With the initial position and destination configured, an intelligent NPC can operate a safe but non-deterministic autopilot (e.g., to actively avoid collisions with other vehicles or choose a shorter path). Hence, when a surrounding vehicle is eliminated, the motions of intelligent NPCs would probably change, causing inaccurate results of what-if reasoning. To alleviate this issue, we try to record the exact moving behaviors (i.e., trajectory waypoints: $\{t_0 : GPS_0(x_0, y_0, z_0), t_1 : GPS_1(...), ...\}$) of each intelligent NPC. As such, we can curate a semantically equivalent accident scenario where each NPC has stable motions without any non-deterministic runtime behaviors. Fortunately, existing ADS simulation platforms all provide user-friendly APIs (e.g., *3D Ground Truth Sensor* of LGSVL [47] and *carla.Vehicle* of CARLA [26]) to help retrieve the NPC status (e.g., position and speed) during runtime. As such, we can then adjust the configurations of the simulation scenario, to replace intelligent NPCs with normal ones that have fixed moving trajectories.

4.2.3 NPC Pre-Trimming. To speed up the procedure of what-if reasoning, especially for congested driving scenarios (e.g., traffic jams with massive vehicles), we seek to quickly filter out those scenario attributes that are definitely irrelevant to the accident, without leveraging time-consuming scenario replaying for each of them. Here, our key observation suggests that if an obstacle is consistently obstructed by other obstacles within the EGO's sensing perspective, these obstacles would not impact the driving decisions made by ADS (i.e., ADS can hardly receive sensor data that derive from these obstacles as inputs). Building on this insight, as shown in Figure 4, we propose a strategy to identify these obstacles by efficiently analyzing the spatiotemporal correlation between the bounding boxes of each obstacle and the EGO. Specifically, at each timestamp, ADSDx labels obstacles as irrelevant to current ADS driving decisions if they fall outside the ADS sensing range or are obstructed by other NPCs (e.g., NPC-A and NPC-E). Notably, we only classify obstacles as accident-irrelevant if they remain irrelevant to ADS driving decisions throughout the entire scenario execution time and remove them all at once.

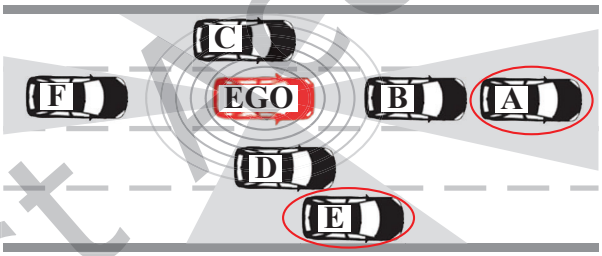


Fig. 4. Example of NPC Pre-Trimming. At this timestamp, NPC-A and NPC-E can hardly influence EGO's driving behaviors as they are strictly blocked by NPC-B and NPC-D.

4.3 Accident-responsible Module Localization

4.3.1 Overview of Diagnosis Methodology. As formalized in §4.1, we can therefore investigate whether a perfectly functioning module (i.e., to replace the original ADS module states with corresponding perfect module states) can help avoid the accident. If not, this module should not be the one primarily responsible for the observed accident. Obviously, the faulty module localization relies heavily on the frequent injection of perfect module states. Therefore, we now introduce the procedures for modeling, constructing, and injecting perfect states to diagnose the ADS modules.

- **State Modeling.** As defined in §4.1, in a given scenario, a perfect module state refers to the runtime state that fully satisfies the module's functional expectations. Specifically, ❶ the perfect perception state can

Just Accepted

be modeled as the true state of surrounding obstacles (e.g., their actual position and size). ❷ The perfect prediction state is modeled as the real future trajectory of the obstacles. ❸ The perfect planning state refers to a driving path that meets multiple performance expectations, including adherence to vehicle dynamics and effective accident avoidance. As discussed in §3.2, we can not achieve such a perfect module state for the planning module. Hence, we propose a customized strategy for diagnosing the planning module. ❹ The perfect control state refers to vehicle control commands (i.e., brake, throttle, etc.) that make the EGO vehicle strictly follow the planned trajectory. However, crafting perfectly correct control commands is no simple feat, which requires extensive expert knowledge of vehicle control and physical dynamics (e.g., should consider massive environmental characteristics such as the friction forces on the road surface). To overcome this challenge, we feature an approximation solution to model the perfect control state as the EGO vehicle's runtime locations that strictly follow the planned trajectory.

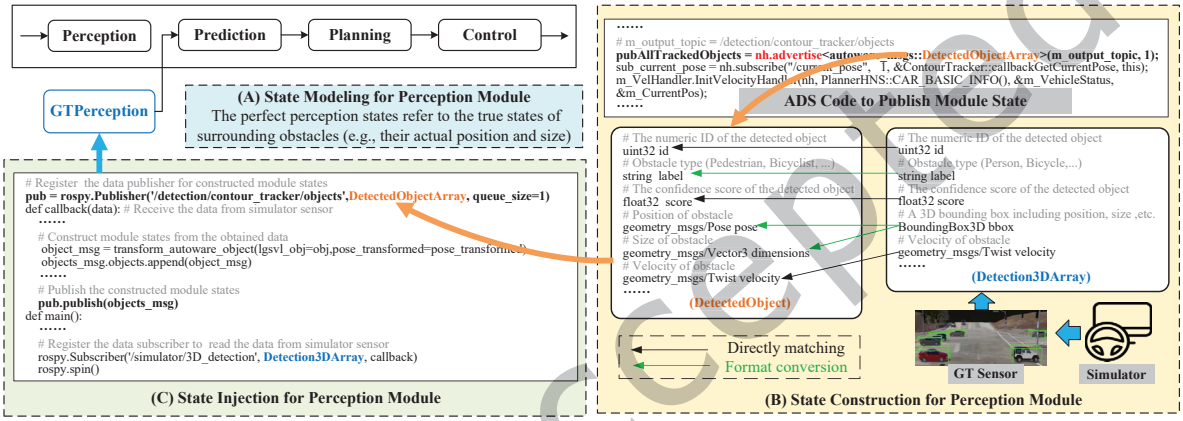


Fig. 5. Running Example of State Modeling, State Construction, and State Injection for the Perception Module of Autoware.

- **State Construction.** As mentioned in §4.1, the constructed perfect module states are both syntactically and semantically correct module runtime states. ❶ To construct syntactically correct states, we should analyze the data structures of the valid module outputs. Fortunately, in industry-grade ADSs with a modular architecture, the outputs of each ADS module are organized and published through consistent framework APIs (e.g., *Node::CreateWriter()* for Cyber-RT [2] and *ros::NodeHandle::advertise()* for ROS [11]). Hence, as shown in Figure 5 (B) after identifying the callsites of these specific APIs, we can easily obtain the syntactic formats of module outputs. ❷ To construct semantically correct states, we obtain the relevant data from two sources: the simulation environment and the internal outputs of the ADS modules. For the perception and prediction modules, we leverage the simulation platform APIs (e.g., *3D Ground Truth Sensor* of LGSVL [47] and *carla.Vehicle* of CARLA [26]) to retrieve the real-time states of surrounding obstacles from the simulation environment, which allows us to construct the perfect perception and prediction module states. For the control module, we monitor the ADS internal communications to obtain the real-time output from the planning module (i.e., the planned trajectory), the primary input of the control module, which enables us to construct the perfect control state (i.e., the EGO vehicle's real-time locations).
- **State Injection.** Within a modular architecture ADS, the communication between different modules is achieved through the ADS middleware framework (e.g., Cyber-RT [2] and ROS [11]). These frameworks commonly enable asynchronous communications between ADS modules through the publisher-subscriber

Just Accepted

mechanism. To be specific, the framework provides APIs to create in-memory data pools (also called as "topics"). As such, the upstream module works by publishing its outputs to a specific topic, and the downstream module would continuously fetch messages from the corresponding topic. Owing to this mechanism, the downstream module would not verify whether the message comes from the correct upstream module or not, once the messages have the correct syntactic formats. Therefore, we can achieve state injections by disabling the upstream module during runtime and creating a virtual publisher to publish messages to the target topic, without any system code modification or re-compiling. Figure 5(C) illustrates an example of state injection for the prediction module of Autoware [6].

- **Running Example.**Figure 5 presents a detailed example of modeling, constructing, and injecting perfect perception states into the Autoware perception module. We begin by modeling the perfect perception states as the real-time states (e.g., positions, orientations, and sizes) of surrounding obstacles in the simulated environment. Next, we identify the relevant code for publishing perception states in Autoware by locating the use of the *ros::NodeHandle::advertise()* API, from which we extract the specific data format (i.e.,*DataObjectArray*). Using the simulator's API (e.g., *3D Ground Truth Sensor* of LGSVL [47]), we then obtain the ground-truth states of obstacles and convert them into semantically equivalent Autoware perception messages. Finally, we disable the original perception module and replace it with a custom data publishing node (as illustrated in Figure 5(C)), which injects the constructed perfect perception states into downstream ADS modules.

4.3.2 Diagnostic Order for ADS Modules. After faithful modeling of perfect module states, we generally follow a sequential order to diagnose each ADS module (i.e., from the upstream module of the driving pipeline to the downstream one), so as to eliminate the influence of error propagation across ADS modules. However, the most challenging point in the above procedure is that one cannot model the perfect state for the planning module. Hence, after confirming the innocence of the perception module and the prediction module, we alternatively turn to diagnosing the control module rather than the planning module. The rationale behind this is that we can boldly assume that the planning module has generated a safe path. Based on this assumption, if a perfectly working control module can help avoid the accident, the control module should be the faulty one. If not, it indicates that the planning module produces an unsafe path, prompting us to diagnose the planning module.

4.3.3 Diagnosis of Perception Module. To construct and inject the perfect perception states into the target ADS, we first dynamically query the real-time ground-truth status of obstacles based on the simulator APIs. With these concrete status values in hand, we rely on manually implemented value transformation rules to make them satisfy the syntactic requirements of ADS (e.g., simulator and ADS use different coordinate systems for describing positions). Finally, these values are published to the downstream ADS module for diagnosis purposes. In particular, the frequency of value injection should be consistent with that of the original perception module (e.g., 0.1s).

4.3.4 Diagnosis of Prediction Module. Given a simulation scenario, we follow a two-step methodology to construct and inject precise prediction states into the target ADS. ❶ **Step-I:** Different from the state injection for the perception module, here, we should first utilize offline scenario replaying to capture the complete trajectories of all obstacles at each timestamp. Only in doing so can we precisely determine the future trajectory at any given timestamp *t*. ❷ **Step-II:** After the off-line value collection, we then re-run the scenario with injected states. Since the injection is quite time-sensitive, we should carefully align the timestamp of the simulator with that of the target ADS, otherwise we would inject imprecise semantics. To ensure timestamp consistency, we stop the simulation until the simulator receives the first communication message from the target ADS (i.e., target ADS activated).

Just Accepted

4.3.5 Diagnosis of Control Module. To achieve the injection of perfect control states, after obtaining the planned trajectories (i.e., outputs of the planning module) at each timestamp t, we force the EGO vehicle to drive exactly along this trajectory, and at the same time, ensure the proper functioning of the target ADS. We follow two key steps to achieve this goal: ❶ **Step-I:** In each time cycle for the publication of control messages, we request the latest planned trajectory from the planning module, used as data references for the EGO movement in the simulation world. ❷ **Step-II:** However, the frequency of control message publication (marked as T) might not be consistent with that of trajectory planning (Δt in the above formula). This kind of frequency inconsistency would make it hard to directly extract the EGO position from the planning outputs at timestamp t. To tackle this issue, we should leverage reliable interpolation algorithms to estimate the exact EGO position at timestamp t. Here, we resort to the built-in interpolation APIs of our target ADSs (i.e., *InterpolateUsingLinearApproximation* in Apollo [1] and *lerp* in Autoware [6]) to implement the prototype of ADSDx. ❸ **Step-III:** Finally, we accordingly move the autonomous vehicle in the simulation world, and notify the ADS system with updated localization information.

4.3.6 Diagnosis of Planning Module. The planning module aims to create a safe trajectory that can reach the destination without any accidents. It is theoretically infeasible to determine a perfectly planned trajectory. But fortunately, after all the above diagnosis efforts (i.e., perception, prediction, and control), we can already confirm that the accident is either caused by the flaws of the planning module, or the accident is unavoidable (i.e., there are no accident-free trajectories to the destination and the target ADS is not responsible for the accident). Here, our key idea is that we can calculate the drivable areas of the EGO at the simulator side, to determine whether there exists a safe trajectory. The drivable areas (marked as $\mathbb{DA}$) refer to the collection of road regions that the EGO vehicle can reach estimated from its states at a given initial time (i.e., T_{start}), projected over a future time horizon (as depicted in Figure 6). The determination of these areas is subject to the dual constraints of vehicle dynamics and accident avoidance. To express it formally, the drivable areas of the EGO at each timestamp t ($T_{start} \leq t$) can be depicted as a set of polygonal areas (denoted as $poly_i$).

$$\mathbb{DA}(\mathbb{S}, T_{start}, t) = \{\text{poly}^1, \text{poly}^2, \dots, \text{poly}^m\}$$

To practically compute the drivable areas without violating vehicle dynamics, we utilize the algorithm (denoted as f) proposed in [53], which employs a simplified vehicle model and a conservative approximation of the reachable areas of EGO. In this algorithm, the drivable areas at timestamp t are efficiently calculated by propagating from the drivable areas at the previous timestamp, i.e., $\mathbb{DA}(\mathbb{S}, T_{start}, t) = f(\mathbb{DA}(\mathbb{S}, T_{start}, t - \Delta t), \Delta t)$. Furthermore, depending on the specific accident types, the preliminary drivable area should be further restricted and pruned to guarantee a accident-free path. Therefore, we adjust the calculation of the drivable areas as follows:

$$\mathbb{DA}(\mathbb{S}, t_s, t) = F(f(\mathbb{DA}(\mathbb{S}, t_s, t - \Delta t), \Delta t), Area_{Forbidden})$$

$$F(\mathbb{DA}, Area_{Forbidden}) = \{X \mid X \in \mathbb{DA} \wedge \forall Y \in Area_{Forbidden}, X \cap Y == \emptyset\}$$

In this context, $Area_{Forbidden}$ represents the restricted area specified by a given accident type. Formalizing these restricted areas allows the diagnostic process to be tailored to different accidents. We subsequently detail the modeling approach for several common accident types:

- **Forbidden Areas for Collision And Immobility:** For collision and immobility scenarios, our method filters out lanes that are unreachable by the EGO vehicle based on road connectivity. As illustrated in Figure 6, the drivable area in oncoming lanes is pruned according to the planned route and solid lane markings. Consequently, if a drivable area persists after this refinement, it indicates that a superior planning trajectory existed which could have prevented the collision or obstruction.
- **Forbidden Areas for Traffic Violation:** For traffic violations, the restricted area defined for collision accidents can be further extended by incorporating areas prohibited by traffic regulations. Fortunately,

Just Accepted

existing work [42] for traffic violation testing not only provides detailed violation detection oracles but also supplies regulation-to-area mappings based on rigorous modeling. VioHawk [42] decomposes traffic regulations into driving intentions and driving conditions, modeling each as concrete road areas. For example, red-light regulation forbidden areas, can be represented by the intersection the traffic light located, while a yield regulation can be modeled as the areas preceding the NPC vehicles. Although our primary experiments focus on collision scenarios, a case study in §7 demonstrates the feasibility of this adaptation.

To sum up, the planning module is deemed responsible for the accident if there exist safe drivable areas for the EGO vehicle across all timestamps (i.e., $\forall t \in (t_{\mathcal{T},T_c}], \mathbb{DA}(\mathbb{S}, t_{\mathcal{T}}, t)! = \emptyset$).

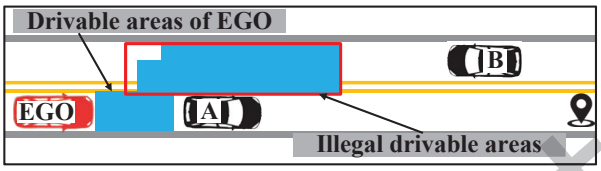


Fig. 6. Drivable Area Calculation in Lane-borrow Scenario.

5 Evaluation

5.1 Experiment Setup

5.1.1 Research Questions. To evaluate the effectiveness and utility of ADSDx, we design experiments to answer the following research questions:

- **RQ1:** How effective is ADSDx in identifying accident-triggering conditions? (§5.2)
- **RQ2:** How effective is ADSDx in identifying the accident-responsible module? (§5.3)
- **RQ3:** How efficient is ADSDx in diagnosing ADS-involved accidents? (§5.4)

5.1.2 Target ADSs. Recently, two open-source ADSs, namely Apollo [1] and Autoware [6], have been gaining increasing attention in the industry. Both offer a comprehensive set of capabilities for high-level autonomous driving and have emerged from the test stage to commercial applications. Besides, they have been selected as evaluation targets in various research works [19, 20, 34, 36, 40, 56, 58, 63, 68]. Hence, we evaluated the stable versions of these systems: Apollo 6.0 and Autoware 1.15 (open-planner 2.5).

5.1.3 ADS Simulation Platform. Here, we finally chose LGSVL 2021.2.2 [9] to build our prototypes due to its high fidelity and flexibility. Although the LGSVL ceased remote services in January 2022, the stable local version of LGSVL has already been available [13] and many SOTA works [25, 42, 55] still use LGSVL to develop prototypes due to its excellent testing support for mainstream ADSs like Apollo and Autoware.

5.1.4 Baselines. To the best of our knowledge, there is currently no directly comparable, off-the-shelf tool that enables an end-to-end comparison with our proposed system. Therefore, we selected SOTA tools as baselines for the two key techniques of ADSDx separately. ❶ For accident-triggering conditions inference, we selected the scenario simplification tool [16] (denoted as `Less_Is_More`), as a baseline for comparison. `Less_Is_More` aims to reduce scenario complexity by replaying scenarios and removing attributes that do not affect the EGO vehicle's trajectory. Additionally, I created an ablation version of ADSDx, referred to as ADSDx$_{\triangle 1}$ (i.e., ADSDx without scenario transformation), to highlight the unique benefits of our scenario transformation. ❷ For accident-responsible module localization, we selected three SOTA diagnostic tools referenced in §2.3: ACAV [55],

Just Accepted

ROCAS [27], and DVCA [60]. As discussed in the same section, DiaVio [46] is excluded from our comparison due to its fundamentally different technical scope. Since ROCAS and DVCA do not offer publicly available implementations, we reimplemented them based on the methodologies described in their respective publications.

5.1.5 Dataset of ADS-involved Accidents ($Dataset_{ACC}$). To evaluate the performance of ADSDx, we need to construct a dataset of ADS-involved accidents (i.e., accident scenarios of our target ADSs). Notably, for each accident scenario, its ground-truth fault-triggering conditions and fault-inducing module should be faithfully annotated. Inspired by ACAV [55], we chose to intentionally inject faults into the specific ADS modules to curate our benchmark dataset (i.e., $Dataset_{ACC}$). In such a manner, we can correlate the accidents and specific faulty modules by the fault injected point and leverage the fault causality of injected code segments to infer the fault-triggering conditions within the accident scenarios, ensuring the objectivity of our dataset annotations.

Specifically, we extended the injected faults provided by ACAV [55] to 22 faults covering the perception, prediction, planning, and control modules of both Apollo and Autoware. Using an ADS testing framework (i.e., DriveFuzz [36]), we conducted extensive tests on Apollo and Autoware, lasting over 60 CPU hours, and recorded the collision scenarios. At last, we selected 81 collision scenarios from the detected collisions (i.e., 40 for Apollo and 41 for Autoware) to form our collision dataset. These scenarios meet the following criteria: ❶ they do not occur in the origin ADS versions (i.e., the version without any fault injection), ❷ they can be reliably reproduced and replayed, and ❸ they each exhibit at least one distinct accident features with each other (e.g., differing collision locations or collision angles). Details on how to annotate the ground-truth diagnosis results are clarified in §5.2 and §5.3, respectively. Notably, numerous accident scenarios encountered in ADS testing are not primarily the ADS's responsibility (e.g., the `EGO` vehicle has no space to escape obstacles). To evaluate the diagnostic tool's capability to distinguish such cases correctly, we selected six non-ADS-responsible accident scenarios from the aforementioned test corpus for both Apollo and Autoware.

5.1.6 Execution Environment. All experiments were conducted on a Ubuntu 18.04 server with 314 GB of memory, two Intel Xeon Gold 5215 CPUs, and four NVIDIA GeForce RTX 2080 TIs. This hardware configuration can ensure the stable execution of our target ADSs and the ADS simulation platform.

5.2 Effectiveness of Accident-triggering Condition Inference

5.2.1 Ground Truth Construction. With the faulty code segments in hand, we can faithfully infer the potential fault-triggering scenario attributes by analyzing the fault causality of the injected fault. For example, regarding a prediction fault, which makes "ignore" decisions for all static NPCs near the planned trajectory, its potential fault-triggering scenario attributes involve all static NPCs within the `EGO`'s perception range. Additionally, all scenario objects in the direction of the `EGO`'s movement are also considered potential fault triggers, as they may significantly impact the `EGO`'s actions. Subsequently, we conduct a detailed manual analysis to scrutinize these scenario attributes individually, identifying and marking those truly relevant to the accident as the accident-triggering conditions. To be more specific, we incrementally adjust those scenario attributes to monitor the influence on runtime ADS behaviors. Given a scenario attribute, if reasonable changes of this attribute can reduce the severity of the accident (i.e., reduce collision speed), or even totally avoid the accident, we annotated this attribute as an accident-triggering one. To avoid biases, each accident is double-checked by two authors, all of whom have over 2-year experience in ADS testing.

5.2.2 Experiment Design. We evaluated the effectiveness of ADSDx as well as the two baselines (i.e., `Less_Is_More` and $ADSDx_{\triangle 1}$) mentioned earlier in inferring the accident-triggering conditions for an ADS-involved accident, by comparing the reported accident-relevant attributes with the ground-truth ones. To evaluate the performance of different tools, we measured the precision and recall of accident-triggering condition inference and recorded the proportion of accident-irrelevant NPC vehicles remaining after diagnosis.

Just Accepted

Table 1. Effectiveness of Accident-triggering Condition Inference.

Tools	Precision (Avg.)	Recall (Avg.)	# of NPC$_{relevant}$ (Avg.)
ADSDx	95.0%	100.0%	3.0/20.2 (14.9%)
`Less_Is_More`	18.5%	94.3%	16.0/20.2 (79.2%)
ADSDx$_{\triangle 1}$	39.9%	100.0%	8.8/20.2 (43.6%)

ADSDx$_{\triangle 1}$: ADSDx without scenario transformation;
# of NPC$_{relevant}$: number of accident-relevant NPCs.

5.2.3 Experiment Results. As shown in Table 1, ADSDx has high precision (i.e., 95.0%) and recall (i.e., 100%) in identifying accident-triggering conditions. Compared to the two baselines, ADSDx demonstrated a precision improvement of 55%~76.5%, indicating its superior ability to effectively filter out accident-irrelevant scenario attributes. In the following, we conducted detailed breakdown analysis to demonstrate the limitations of each tool.

- **Breakdown Analysis of ADSDx$_{\triangle 1}$.** The results show that the scenario transformation can practically improve the precision of accident diagnosis by filtering out an additional 28.7% accident NPCs. After careful manual case studies, we confirmed that it is mainly because there exist intelligent NPCs declared in the simulation files of over 50% accident scenarios in *Dataset$_{ACC}$*. Without scenario transformation to eliminate their influences, one can hardly pinpoint the accident-relevant scenario attributes through what-if reasoning.
- **Breakdown Analysis of `Less_Is_More`.** Similar to ADSDx$_{\triangle 1}$, `Less_Is_More` lacks scenario transformation, resulting in the failure to remove many accident-irrelevant NPCs. Additionally, during the scenario replay process, `Less_Is_More` only considers the absolute value of the relative speed between the `EGO` vehicle and the colliding vehicle (with a threshold difference of 0.1 m/s) to differentiate similar collisions. This condition is overly strict, causing some similar collisions to be classified as different, thus retaining more accident-irrelevant attributes. In contrast, ADSDx accounts for the relative speed relationship, rather than the absolute value, to reduce false positives. Furthermore, to minimize false negatives, ADSDx incorporates additional feature dimensions more feature dimensions (e.g., collision location) than `Less_Is_More` does.
- **Breakdown Analysis of ADSDx.** We also manually investigated all the false positives (FP) reported by ADSDx. The primary reason lies in that, when checking the consistency of collision features (i.e., *accident_features* mentioned in §4.1), we rely on the simulator's internal collision detection callback function to obtain the collision object's state (e.g., position and orientation). However, the data precision provided by this callback function may be insufficient, leading to fluctuations in collision features that are on the boundary between different categories (e.g., a collision occurring at the intersection of multi-lane roads, or the collision angle being between a side impact and a head-on collision). For example, in an intersection accident caused by prediction issues, the collision position may lie exactly at the junction of the straight and turning lanes, causing the same collision to be recorded in different lanes during different scenario executions. This can lead ADSDx to mistakenly believe that the original collision was avoided, thus incorrectly classifying accident-irreverent scenario attributes as accident-triggering conditions. Although comparing these collision features instead of only monitoring the occurrence of collision may inevitably result in some FPs, this cautious collision detection greatly helps improve the recall of our diagnosis (i.e., ADSDx did not miss any accident-relevant scenario attributes).

Just Accepted

5.3 Effectiveness of Accident-responsible Module Localization

5.3.1 Ground Truth Construction. Since we utilize fault injecting to curate $Dataset_{ACC}$, we directly annotated the module where the fault was introduced as the ground-truth accident-responsible module for each related accident. Table 2 presents the details of the ground-truth data.

Table 2. Statistics of Ground-truth Accident-responsible Module.

Target ADS	# of Accident Scenarios Caused by Accident-responsible Modules					
	Perception	Prediction	Planning	Control	Non-ADS-responsible	All
Apollo	6	6	18	10	6	46
Autoware	14	9	8	10	6	47

5.3.2 Experiment Design. In this section, we evaluate the reliability of our state injection and the effectiveness of ADSDx in automatically identifying the accident-responsible ADS module. To assess the reliability of state injection, we recorded the message delay sequences of key ADS modules before and after each state injection. We then used statistical significance testing [15] to evaluate whether the state injection caused significant delays in the ADS module states. After that, we compared ADSDx with three baselines mentioned in §5.1 (i.e., ACAV [55], ROCAS [27], and DVCA [60]) in $Dataset_{ACC}$. Notably, ACAV aims to identify all possible fault events and might report multiple potential faulty modules. Hence, to align the experimental setup, we refer to the way used in the ACAV's publication, which designated the module associated with the longest-lasting event as the accident-responsible module identified by ACAV. Additionally, ACAV, ROCAS, and DVCA only support accident diagnosis on Apollo. For example, the diagnosis of ACAV highly relies on customized fields in Apollo (e.g., the obstacle priority field within prediction messages and the behavioral planning field within planning messages). As a result, we only evaluate these three baselines on the Apollo-oriented accidents in $Dataset_{ACC}$.

Table 3. The Statistical Significance Test Results for ADS State Injection.

State Injection	Statistical Significance Test p-value (MIN~MAX(AVG))			
	Latency($State^{Perc}$)	Latency($State^{Pred}$)	Latency($State^{Plan}$)	Latency($State^{Cont}$)
$\mathbf{GT}_{Perc}$	0.60~0.94 (0.82)	0.65~0.97 (0.83)	0.73~0.83 (0.78)	0.38~0.94 (0.75)
$\mathbf{GT}_{Pred}$	✗	0.51~0.99 (0.74)	0.45~0.89 (0.61)	0.29~0.94 (0.62)
$\mathbf{GT}_{Cont}$	0.71~0.98 (0.91)	0.23~0.93 (0.60)	0.14~0.94 (0.61)	✗

A p-value ≥ 0.05 indicates that the state injection makes no significant latency effect on ADS messages.
✗ means this module state is not generated under state injection.

5.3.3 Experiment Results. Table 3 presents the p-values from the significance tests for the message delay sequences of each ADS module state before and after state injection. A p-value greater than 0.05 indicates no significant difference in these two sequences. The results show that state injection does not cause any significant delays in any ADS module, as all p-values are above 0.14.

Just Accepted

Table 4. Effectiveness Results of Accident-responsible Module Localization.

Tools	Target ADS	Diagnostic Accuracy					
		Acc(Per.)	Acc(Pre.)	Acc(Plan.)	Acc(Con.)	Acc(Non.)	Acc(All)
ADSDx	Apollo	6/6(100.0%)	6/6(100.0%)	17/18(94.4%)	10/10(100.0%)	6/6(100.0%)	45/46(97.8%)
	Autoware	14/14(100.0%)	9/9(100.0%)	6/8(75.0%)	10/10(100.0%)	5/6(83.3%)	44/47(93.6%)
ACAV	Apollo	0/6 (0.0%)	2/6(33.3%)	3/18(16.7%)	9/10(90.0%)	1/6(16.7%)	15/46(32.6%)
ROCAS	Apollo	1/6(16.7%)	2/6(33.3%)	10/18(55.6%)	8/10(80.0%)	0/6 (0.0%)	21/46(45.7%)
DVCA	Apollo	4/6(66.7%)	4/6(66.7%)	13/18(72.2%)	7/10(70.0%)	0/6 (0.0%)	28/46(60.9%)

Acc (.) represents the accuracy for diagnosing accidents caused by each ADS module;

Table 4 presents the overall results of accident-responsible module localization. Generally, ADSDx achieves a high accuracy of 96.3% in identifying the accident-responsible ADS module that is primarily responsible for the observed accident. In contrast, the three baselines achieved accuracies ranging from 41.2% to 70.0% for the accident scenarios they support, which is significantly lower than that of ADSDx. Additionally, the results also reveal that ACAV, ROCAS, and DVCA hardly distinguish the accidents where the ADS is not at fault. As described below, we also conducted a thorough breakdown analysis of the inaccurate diagnostic results of each tool to understand their limitations.

- **Breakdown Analysis of ACAV.** A primary limitation of ACAV is that its design fundamentally assumes a fault-free perception module. Most of the inaccurate diagnostic results of ACAV occurred because it misdiagnosed planning-responsible accidents and prediction-responsible accidents as control-responsible accidents. The underlying reason is that ACAV relies on state deviations to identify control events (i.e., the difference between the ADSDx's actual trajectory and the planned trajectory). As discussed in §2.3, such a mechanism is unreliable since not all state deviations will finally result in accidents. Further analysis of these accidents revealed that when predictions and modules fail, the ADS tends to make more reckless driving decisions (e.g., increasing speed to overtake), resulting in the control event (ie., a vehicle going out of control) becoming more prominent and often overshadowing other events in duration. Additionally, when ignoring the control events, ACAV's accuracy for accidents caused by prediction and planning faults significantly improved to 83.3%. These results are close to the findings mentioned in the ACAV literature, as we injected the same faults into Apollo's prediction and planning module as ACAV did.
- **Breakdown Analysis of ROCAS.** ROCAS identifies faulty modules through delta testing by comparing pairs of similar scenarios with opposing collision outcomes. The key insight is that the earliest differences in module states between the two scenarios are responsible for collision avoidance. However, in many incorrect diagnostic results of ROCAS, we found that collision avoidance was not caused by differences in ADS states, but rather by slight perturbations introduced by delta, which disrupted the original collision conditions (e.g., the colliding object was moved from its initial position). Moreover, this approach is predicated on the implicit assumption that the under-test accident must be caused by a fault within the ADS modules. Consequently, it cannot diagnose non-ADS-responsible accidents present in the dataset. Additionally, we also found that unrigorous heuristic hyperparameters caused some misdiagnosed cases. For instance, ROCAS assumes that the difference of the upstream module must occur no earlier than 3 seconds before the downstream module's divergence to be considered the cause of the downstream module's difference. However, in certain scenarios, perception and prediction differences may emerge

Just Accepted

earlier but only trigger different downstream module states when the NPC is in close proximity to the EGO vehicle. In contrast, ADSDx provides a more rigorous oracle for distinguishing the relationship between module states and accidents.

- **Breakdown Analysis of DVCA.** As introduced in §9, DVCA's counterfactual causality analysis focuses solely on diagnosing system-side issues, neglecting the impact of system-side message changes on the scenario side (e.g., the behavior of intelligent NPCs in the scenario). Specifically, DVCA's injection of system-side messages indirectly alters NPC driving behavior by influencing the EGO's driving actions. In cases of diagnostic errors, DVCA is unable to accurately determine whether a collision avoidance is due to message injection or the indirectly altered NPC driving behaviors.Morever, the DVCA is weakened by a foundational presupposition: it presumes that in the absence of faults in perception, prediction, and control, the accident should be attributed to the planning module, leading to an unwarranted causal attribution. Furthermore, DVCA currently detects collisions at a coarse granularity, considering only the type of collision objects but not differentiating between detailed features such as collision location, angle, or speed. This limitation results in situations where, in some accident scenarios, even if the message injection helps avoid the initial collision, new accidents may occur, potentially obscuring the outcome. In contrast, the what-if reasoning provided by ADSDx, with its scenario-side diagnostics (e.g., scenario transformation), more rigorous system-side diagnostics (e.g., the drivable area-based planning diagnosis) as well as finer-grained collision feature detection, helps eliminate these unreliable situations.
- **Breakdown Analysis of ADSDx.** ADSDx misdiagnosed 3 planning-responsible accidents as prediction-responsible accidents. For example, in the accident shown in Figure 7, a fault injected into the planning module caused EGO's delay braking decision, resulting in a collision with NPC-A. Interestingly, we found that a perfect prediction module could also help avoid this accident. The reason lies in the fact that the injected faults behind these accidents are only triggered by inaccurate perception or prediction results. In real-world planning, the planning module typically cannot always obtain perfect perception or prediction results, leading to some conservative fault-tolerant mechanisms (e.g., safety distance redundancy). In our scenarios, the injected faults caused the planning module to disregard such mechanisms (e.g., reducing the safety distance threshold) and adopt more reckless driving strategies. Hence, in our diagnostics, the perfect prediction mitigated the impact brought by these faults, thus avoiding collisions. Consequently, ADSDx stopped diagnosing after identifying the prediction as an accident-responsible module, failing to detect the downstream module faults.We will discuss this as a limitation in §8.

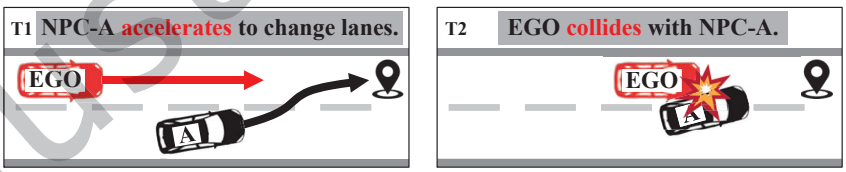


Fig. 7. Example of Misdiagnosed Accident: EGO incautiously avoid the lane changing NPC-A, causing a side-on collision.

Just Accepted

5.4 Efficiency of ADSDx

5.4.1 Experiment Design. We recorded the rounds of simulations and the corresponding time costs required for ADSDx to demonstrate its efficiency. We also compared ADSDx with ADSDx$_{\triangle 2}$ (i.e., ADSDx without NPC pre-trimming), to show the benefits of the NPC pre-trimming in enhancing efficiency.

Table 5. Efficiency Results of ADSDx.

Tools	Rounds of Simulations (Avg.)	Time Cost	
		Avg.	Max.
ADSDx	15.6	347.2s	929.4s
ADSDx$_{\triangle 2}$	24.5	540.3s	1666.0s

ADSDx$_{\triangle 2}$: ADSDx without NPC pre-trimming;

5.4.2 Experiment Results. Table 5 shows that the average time cost of ADSDx is within 6 minutes. As an offline accident diagnostic tool, this level of analysis overhead is acceptable. Furthermore, the results also demonstrate that NPC pre-trimming can boost the efficiency of ADSDx. Statistically, given an accident scenario in $Dataset_{ACC}$, the NPC pre-trimming can help quickly filter 50.7% accident-irrelevant NPCs on average without time-consuming scenario replaying.

6 Utility Study: Fault-relevant Messages Reduction

To further demonstrate the utility of our work, we leveraged ADSDx to perform fault-irrelevant messages reduction, which is the essential task for investigating the in-depth code-level root causes of ADS-involved accidents.

6.1 Motivation

The in-depth root cause analysis of ADS faults is essential as a mitigation method, which pinpoints buggy implementations or improper system configurations triggered by suspicious inputs. It typically requires a detailed examination of the code along the program execution paths relevant to suspicious inputs. However, given the real-time requirements of ADS and its modular architecture, each module processes new inputs in each decision cycle (e.g., 0.1 to 0.2 seconds), resulting in a substantial volume of messages that need to be analyzed. Hence, it is crucial to reduce the fault-irrelevant messages for in-depth root cause analysis. In light of this, we demonstrated the utility of ADSDx in assisting in-depth root cause analysis by measuring the extent to which ADSDx filters out the fault-irrelevant messages.

6.2 Our Methodology

Based on the diagnostic capabilities of ADSDx, we follow the steps outlined below to systematically reduce ADS fault-relevant messages. Firstly, prior to the diagnosis, potential fault-relevant messages include the real-time module input for all ADS functional modules (i.e., perception, prediction, planning, and control) during the scenario execution. Then, through accident-trigger conditions inference and accident-responsible module localization of ADSDx, we can obtain a trimming scenario (i.e., the scenario which removes all accident-irrelevant scenario attributes and time frames) and the corresponding accident-responsible faulty module. With these diagnostic results, we can re-run the trimming scenario and record the input messages of the accident-responsible module as the final fault-relevant messages.

6.3 Dataset

To showcase the utility of ADSDx in assisting in-depth root cause analysis, we collected a new dataset of accident scenarios caused by inherent faults (rather than the artificial bugs in §5) of Apollo and Autoware. With such a dataset, we can demonstrate the benefits brought by ADSDx in uncovering unknown errors within the ADS.

Just Accepted

Specifically, we use the same fuzzing tool in §5 to test the original version of Apollo 6.0 and Autoware1.15+OpenPlanner2.5. After 60 CPU hours, we finally obtained 38 unique collisions to form our dataset (i.e., 20 for Apollo and 18 for Autoware).

6.4 Study Design.

Here, to demonstrate the unique benefits of ADSDx in fault-relevant messages reduction, we recorded the size and number of all potential fault-relevant messages before and after the diagnosis of ADSDx. Additionally, we compared ADSDx with ACAV to highlight its superiority. In particular, we record the real-time inputs corresponding to all suspicious faulty modules reported by ACAV as the fault-relevant messages after ACAV's diagnosis.

Table 6. Results of Fault-relevant Messages Reduction.

Target ADS	# of Fault-relevant Messages (Avg.)					Size of Fault-relevant Messages (Avg. (MB))				
	Original	ADSDx$_{\triangle M}$	ADSDx$_{\triangle T}$	ADSDx	ACAV	Original	ADSDx$_{\triangle M}$	ADSDx$_{\triangle T}$	ADSDx	ACAV
Apollo	417.6	262.6 (↓37.1%)	213.3 (↓48.9%)	116.9 (↓**72.0%**)	277.1(↓33.6%)	36.7	15.5 (↓57.9%)	10.9 (↓70.3%)	4.0 (↓**91.4%**)	27.5(↓27.5%)
Autoware	1308.7	879.9 (↓32.8%)	456.6 (↓65.1%)	313.7 (↓**76.0%**)	✗	13.7	6.5 (↓52.8%)	3.1 (↓77.5%)	1.7 (↓**87.9%**)	✗

ADSDx$_{\triangle M}$ refers to ADSDx without the accident-responsible module localization; **ADSDx**$_{\triangle T}$ refers to ADSDx without the accident-triggering condition inference.

6.5 Study Results

Table 6 shows that ADSDx can significantly filter out 87.9% ~ 91.4% fault-irrelevant messages. Through ADSDx diagnostics, we typically need to analyze only 1.7MB ~ 4MB of data to determine the root cause of an accident. In contrast, even after ACAV diagnostics, an average of 27.5MB of data remains, making it hard to reduce the manual analysis efforts. Our analysis revealed that ACAV reported numerous fault-irrelevant events, each involving two or three potential faulty modules when diagnosing these accidents. Consequently, ADSDx significantly outperforms ACAV by effectively eliminating inputs from non-faulty modules, resulting in a greater reduction of fault-irrelevant messages. Furthermore, ADSDx's ability to filter out accident-irrelevant scenario attributes substantially decreases the message size. By minimizing fault-relevant inputs to a manageable level, ADSDx facilitated the identification of 10 distinct faults in Apollo and Autoware, as shown in Table 7. Notably, we have developed 5/10 patches to mitigate these identified faults. After reporting these results to the ADS developers, 6 out of 10 root causes have been confirmed, and 1 out of 5 patches has been accepted.

7 Case Study: Adaptation for Diagnosing Other Traffic Accidents

This section details the process of adapting ADSDx to diagnose different accident types through two concrete case studies. We selected the following traffic violations, identified by prior work [42], as examples. In **Case-I**, the EGO vehicle illegally crosses double yellow lines to overtake another NPC vehicle. In **Case-II**, the EGO vehicle fails to yield to an NPC vehicle that arrives at the intersection earlier.The adaptation for these cases does not require major architectural changes but involves two key modifications:

- **Accident Detection Oracle**: The core function for detecting the accident is replaced with traffic violation-specific oracles from existing work [42]. Furthermore, during scenario replay, we also adjusted ADSDx's original checks for accident location and involved objects (e.g., the vehicle being overtaken) based on the characteristics of each violation.
- **Drivable Area Calculation**: As outlined in §4.3, the restricted areas for these violations are identified using the regulation-to-area mappings from VioHawk [42]. The specific area mappings for each case are

Just Accepted

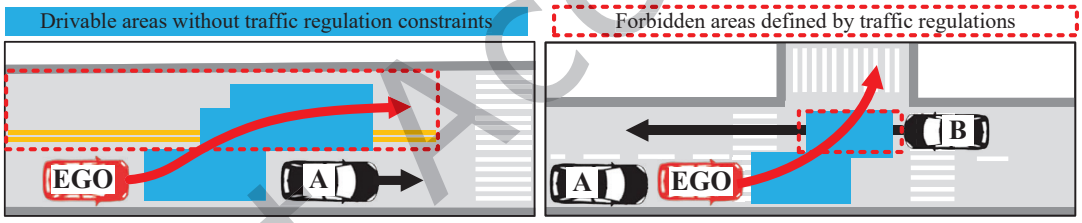
Table 7. Identified Root Causes on Apollo and Autoware.

	ID	Module	Root Cause Description	Confirmed	Patched
Apollo	R1	Planing	No safety operations for correcting overtaking-failures (e.g., go back to origin lane)		
	R2	Prediction	Improper speed threshold for identifying static NPCs	⊠	◐
	R3	Planning	Totally ignoring the motions of the trailing NPCs		
	R4	Prediction	Unrobustness of AI models used for prediction		
Autoware	R5	Control	Frequency mismatch between EGO status updating and trajectory planning	⊠	◐
	R6	Planning	Totally ignoring the motions of the trailing NPCs		
	R7	Planning	Publication of planned path is not allowed when the previous one can reach the destination	⊠	◐
	R8	Planning	Not verifying if the planned path is on valid lanes	⊠	◐
	R9	Planning	Imprecise prediction of the collision position	⊠	●
	R10	Control	Ineffective control algorithms in sharp turns	⊠	

: Root cause confirmed by the developers; ◐: Patch submitted, waiting for the code review; ●: Patch accepted by the developers;

shown in Figure 8. In **Case-I**, the traffic-regulation-restricted area refers to the oncoming lane separated by the double yellow lines. In **Case-II**, the restricted area is defined as the areas preceding NPC-B (the NPC vehicle that entered the intersection before the EGO vehicle).

ADSDx's diagnostic analysis attributed **Case-I** to a planning module failure**Case-II** to a prediction module failure. Additionally, we have included visual demonstrations of the **Case-I** as an example within our artifact [14], illustrating the scenario attributes before and after the scenario-side diagnosis.



(A) Case-I: EGO illegally crosses double yellow lines to overtake NPC-A

(B) Case-II: EGO fails to yield to NPC-B that arrives the intersection earlier

Fig. 8. Examples of Drivable Area Calculation for Traffic Violations

Just Accepted

8 Discussion

8.1 Limitations

Although our experiments (§5) have demonstrated the effectiveness and utility of ADSDx, some limitations still need further discussion. ❶ **(Limitation to diagnose multi-module faults)** ADSDx currently only reports one accident-responsible module when diagnosing accidents. We admitted that there do exist accidents that can be attributed to multiple ADS modules. However, we argue that ADSDx can identify the first error-prone module in the diagnostic pipeline. In such a manner, one can repeatedly use ADSDx to pinpoint the other error-prone modules after patching the first. ❷ **(Limitation to diagnose real-world accidents)** ADSDx can only diagnose accidents that can be reproduced in a simulation environment, and it is not directly applicable to real-world accidents. To fill this gap, we can resort to existing data-driven techniques, such as those described in [30, 67, 69],

to recreate real-world accidents in a simulated environment. ❸ **(Limitations to diagnose accidents on more diverse scenarios)** We acknowledge that current validation mainly focuses on common road structures (e.g., multi-lane roads, intersections, forks, and merge roads). More complex edge situations, such as building areas and bridges, have not yet been tested. This limitation is related to engineering optimization rather than methodology itself, and we consider it as part of our future work.

8.2 Scalability

In this section, we discuss the scalability of ADSDx in diagnosing different accident types and migrating to various simulation platforms. ❶ **(Scalability when diagnosing different accident types)** The current version of ADSDx primarily focuses on collision accidents (i.e., the most severe and common accident type). However, as demonstrated in §7, we argue that once equipped with existing accident-monitoring oracles (i.e., oracles to observe the occurrence of different accidents) proposed by existing works (e.g., [42, 57, 70]), ADSDx can diagnose more complex accident types like traffic rule violations. ❷ **(Scalability when migrating to different ADSs)** The current ADSDx has implemented prototype tools on two mainstream ADS systems, Apollo [1] and Autoware [6], and supports the primary ADS communication frameworks (Cyber-RT [2] and ROS [11]). When migrating to a different ADS, manual analysis is required to collect the relevant module output data formats for the target ADS and develop data conversion scripts. However, these manual costs are highly manageable for ADS testers and developers and need to be performed only once. ❸ **(Scalability when migrating to different simulation platforms)** The current version of ADSDx relies on a simulation platform (i.e., LGSVL [47]) to replay scenarios and gather necessary environmental data for diagnosis, which include runtime states of dynamic scenario obstacles (e.g., the position and velocity of the EGO and NPCs) and states of the traffic signals. Despite this, the overall design of ADSDx is not limited to any specific simulation platform, as these data are readily accessible across different simulation platforms. To illustrate this, Table 8) documents the built-in APIs from LGSVL [47] and Carla [26] that can be used to obtain these data.

8.3 Threats to Validity

We conducted a detailed discussion of potential threats to validity from the following three perspectives: ❶**Internal Validity.** During the dataset selection and annotation process, subjective judgments influenced by the annotators' experience and potential biases in their understanding of the annotation standards may affect the objectivity of the results. To mitigate this threat, we first expanded and enhanced the data source by referencing the dataset from previously published studies (i.e., ACAV [55]). Furthermore, both authors first discussed and clarified the annotation standards before individually annotating the data, followed by a collective discussion of any discrepancies to further strengthen the objectivity of the annotations. ❷**External Validity.** ADSDx is evaluated in a simulated environment with predefined scenarios. While this allows controlled testing, its

Table 8. The Required Built-in APIs in Simulation Platforms.

Required Data	LGSVL API	Carla API
EGO/NPC's position	*lgsvl.Agent().state.transform.position*	*carla.Vehicle.get_location()*
EGO/NPC's rotation	*lgsvl.Agent().state.transform.rotation*	*carla.Vehicle.get_transform().rotation*
EGO/NPC's scale	*lgsvl.Agent().state.bounding_box*	*carla.Vehicle.bounding_box*
EGO/NPC's velocity	*lgsvl.Agent().state.transform.velocity*	*carla.Vehicle.get_velocity()*
EGO/NPC's angular velocity	*lgsvl.Agent().state.transform.angular_velocity*	*carla.Vehicle.get_angular_velocity()*
traffic light states	*lgsvl.Controllable().current_state*	*carla.TrafficLight.state()*

Just Accepted

performance on commercial ADSs and real-world accident scenarios requires further validation. Nevertheless, we chose to implement and validate the ADSDx prototype on two mainstream industrial-grade ADSs (i.e., Apollo [1] and Autoware [6]). These systems are widely used in both academia and industry, enhancing the representativeness of ADSDx's performance results. Moreover, as discussed in §8.1, ADSDx could be integrated with accident reconstruction techniques in the future to enable diagnostics of real-world accident scenarios. **❸Construction Validity.** During dataset construction, the inherent randomness of the ADS AI models and occasional instability of the simulation environment may lead to unstable accident scenarios, which can compromise the validity of accident labeling and subsequent diagnostic analysis. To mitigate this issue, we excluded accident scenarios that could not be consistently reproduced by running multiple iterations during dataset construction (as discussed in §5.1). Additionally, variations in collision features (e.g., vehicle speed during collisions) during scenario replay may also affect diagnostic results. To address this, we accounted for permissible variations in each collision feature (e.g., collision angle variations within a fixed threshold) instead of directly comparing absolute values.

9 Related Work

9.1 ADS Testing and Accident Diagnosis

Most existing works are devoted to detecting harsh driving scenarios [21, 34–36, 41, 58, 61, 68], while few attention [27, 46, 55, 60] has been paid to the accident diagnosis. Diavio [46] leverages a fine-tuning large language model to identify the responsible parties in accidents (i.e., the primary at-fault vehicle) and accident categories, but it cannot pinpoint both the accident-triggering conditions and system failures within ADS. ROCAS [27] and ACAV [55], take a step further, capable of automatically analyzing ADS-involved accidents to identify potential module-level failures in ADS. However, these methods highly depend on unreliable probability-based inference methodologies, which result in inaccurate diagnosis results hindering further fault confirmation and mitigation. The key insight of the ROCAS is that the earliest occurrence of a message discrepancy identifies the module responsible for the differing outcomes between the accident scenario and its variant (a scenario where no accident occurs). However, this assumption does not always hold in practice, as ROCAS lacks a more reliable analysis to confirm whether message discrepancies are truly related to the accident. ACAV, on the other hand, compares system messages and interactions with key obstacles in the scenario to identify suspicious events (e.g., unsafe driving decisions). Yet, it also lacks deterministic analysis to pinpoint the exact event that is the true root cause of the accident. Morever, some studies [16, 60] employ a modify-replay-observe paradigm like ADSDx does. As discussed later §9.2, this strategy is common not only in ADS-oriented diagnosis but also across other domains, such as network protocols [28, 48, 49] and cyber-physical systems [22]. Although these approaches may seem similar to our "what-if" reasoning at a high level, they are limited to either the scenario side or the system side in their problem modeling. In contrast, ADSDx integrates both scenario and system perspectives, aiming to locate the system's faulty modules and the corresponding scenario-trigger conditions. Notably, ADSDx's modeling is not merely an integration of the two diagnostic dimensions but involves addressing the interdependencies between factors across dimensions. For instance, scenario-side diagnostics can mitigate the impact of system-side diagnostics on scenario attributes. Additionally, we introduce unique key designs on both the scenario and system sides (e.g., scenario transformation, NPC pre-trimming, and drivable area-based planning diagnosis) to ensure the rigor of the "modify-replay-observe" process and the effectiveness of accident diagnosis (e.g., further attributing non-ADS primary responsibility accidents).

9.2 Fault Diagnosis

Fault diagnosis is a critical area of research in both software engineering and security community. Traditional software crash analysis [18, 23, 24, 32, 62] relies on identifying accurate crash locations (captured by sanitizers such as ASAN [50]) and analyzing detailed crash states (e.g., core dumps and Intel PT traces). However, ADS-oriented

Just Accepted

accidents are often caused by logical errors, which lack the crash-related error information that aids diagnosis. To diagnose these logical flaws, common practices [37, 43, 54] suggest focusing on code components where the runtime state deviates from the desired state. However, as illustrated in Figure 2, due to the complexity of ADS functionalities, module state deviations (e.g., actual trajectories diverging from planned ones) are common and do not necessarily correlate with accidents. Other studies [39, 44, 64–66] employ data-driven approaches, analyzing runtime data (e.g., program logs) from systems like microservices [39, 64, 66] and distributed storage systems [65] to identify potential faults. While these methods may help infer abnormal events within ADS modules (similar to ACAV), as discussed in the previous section, such techniques may infer many irreverent events hindering further confirmation. Furthermore, existing works have applied "what-if reasoning" in fault causality analysis for various targets (e.g., network protocols [28, 48, 49], and Cyber-Physical Systems like robotic vehicles (RVs)[22]). Notably, ADSDx is the first to apply "what-if reasoning" to accident diagnosis in high-level, industry-grade ADSs. Among these studies, RVPLAYER[22] is the most similar to ADSDx, as it focuses on diagnosing accidents involving RVs. While both RVPLAYER and ADSDx adopt a high-level state injection approach, the technical challenges and solutions differ significantly due to the complexity gap between ADS and RV. These differences are evident in critical steps such as state acquisition, construction, injection, and validity assurance. For example, sensor data injection in RVPLAYER only requires faithful replay of historical data or control commands, without complex semantic transformations. In contrast, ADSDx's state injection involves rigorous semantic modeling of ADS module states, the acquisition of state-relevant data from multiple sources (including historical simulation results and real-time outputs from upstream ADS modules), and semantically equivalent state transitions.

10 Conclusion

In this work, we propose ADSDx, an automated diagnosis tool for safety-critical accidents of high-level ADSs. Considering the complex nature of ADS accidents, ADSDx leverages what-if reasoning to incrementally narrow down the candidate scope of the accident-triggering driving conditions and pinpoint the faulty ADS module primarily responsible for the accident. In evaluation, we conducted extensive experiments to demonstrate the high effectiveness and utility of ADSDx. With the help of ADSDx, we found 10 safety-critical faults on Apollo and Autoware, 6 of which were confirmed by the developers and 1 patch has been merged into the released version of ADS.

Acknowledgments

We would like to thank the anonymous reviewers for their insightful comments that helped improve the quality of the paper. This work was supported in National Natural Science Foundation of China (62402116). Yuan Zhang was supported in part by the Shanghai Rising-Star Program 210A1400700 and the Shanghai Pilot Program for Basic Research-Fudan University 21TQ1400100 (21TQ012). Min Yang is the corresponding author, and a faculty of Shanghai Institute of Intelligent Electronics & Systems and Engineering Research Center of Cyber Security Auditing and Monitoring.

References

[1] Apollo: An Open Autonomous Driving Platform. https://github.com/ApolloAuto/apollo, 2023.
[2] Apollo CyberRT framework for Autonomous Driving. https://cyber-rt.readthedocs.io/en/latest/, 2023.
[3] Apollo Predcition Issue. https://github.com/ApolloAuto/apollo/issues/14637, 2023.
[4] ASAM OpenSCENARIO. https://www.asam.net/standards/detail/openscenario, 2023.
[5] Autonomous Vehicle Collision Reports. https://www.dmv.ca.gov/portal/vehicle-industry-services/autonomous-vehicles/autonomous-vehicle-collision-reports/, 2023.
[6] Autoware-AI. https://github.com/autowarefoundation/autoware, 2023.
[7] Carla SUMO. https://carla.readthedocs.io/en/latest/adv_sumo/#sumo-co-simulation, 2023.
[8] General Motors. https://www.gm.com/, 2023.

Just Accepted

[9] LGSVL 2021.2.2. https://github.com/lgsvl/simulator/tree/release-2021.2, 2023.
[10] LGSVL NPC Vehicle. https://www.svlsimulator.com/docs/python-api/python-api/#npc-vehicles, 2023.
[11] The Robot Operating System (ROS). https://www.ros.org/, 2023.
[12] Waymo. https://waymo.com/, 2023.
[13] Local Version for LGSVL. https://github.com/emocat/simulator/tree/release-2021.2, 2024.
[14] Artifact of ADSDx: Prototypes, Simulation Platform, and Key Evaluation Results. https://zenodo.org/records/17847407, 2025.
[15] Statistical Significance Test. https://en.wikipedia.org/wiki/Statistical_significance, 2025.
[16] Paolo Arcaini, Xiao-Yi Zhang, and et al. Less is More: Simplification of Test Scenarios for Autonomous Driving System Testing . In *ICST*, 2022.
[17] Andrey Balmin, Thanos Papadimitriou, and et al. Hypothetical Queries in An OLAP Environment. In *VLDB*, 2000.
[18] Tim Blazytko, Moritz Schlögel, and et al. AURORA: Statistical Crash Analysis for Automated Root Cause Explanation. In *USENIX Security*, 2020.
[19] Yulong Cao, S Hrushikesh Bhupathiraju, and et al. You Can't See Me: Physical Removal Attacks on LiDAR-based Autonomous Vehicles Driving Frameworks. In *USENIX Security*, 2023.
[20] Yulong Cao, Ningfei Wang, and et al. Invisible for Both Camera And Lidar: Security of Multi-sensor Fusion Based Perception in Autonomous Driving under Physical-world Attacks. In *SP*, 2021.
[21] Mingfei Cheng, Yuan Zhou, and et al. BehAVExplor: Behavior Diversity Guided Testing for Autonomous Driving Systems. In *ISSTA*, 2023.
[22] Hongjun Choi, Zhiyuan Cheng, and et al. RVPLAYER: Robotic Vehicle Forensics by Replay with What-if Reasoning. In *NDSS*, 2022.
[23] Weidong Cui, Xinyang Ge, and et al. REPT: Reverse Debugging of Failures in Deployed Software. In *OSDI*, 2018.
[24] Weidong Cui, Marcus Peinado, and et al. Retracer: Triaging Crashes by Reverse Execution from Partial Memory Dumps. In *ICSE*, 2016.
[25] Jiarun Dai, Bufan Gao, and et al. SCTrans: Constructing a Large Public Scenario Dataset for Simulation Testing of Autonomous Driving Systems. In *ICSE*, 2024.
[26] Alexey Dosovitskiy, German Ros, and et al. CARLA: An Open Urban Driving Simulator. In *CoRL*, 2017.
[27] Shiwei Feng, Yapeng Ye, and et al. ROCAS: Root Cause Analysis of Autonomous Driving Accidents via Cyber-Physical Co-mutation. In *ASE*, 2024.
[28] Ari Fogel, Stanley Fung, and et al. A General Approach to Network Configuration Analysis. In *NSDI*, 2015.
[29] Sainyam Galhotra, Amir Gilad, and et al. HypeR: Hypothetical Reasoning with What-If And How-To Queries Using a Probabilistic Causal Approach. In *SIGMOD*, 2022.
[30] Alessio Gambi, Tri Huynh, and et al. Generating Effective Test Cases for Self-driving Cars from Police Reports. In *ESEC/FSE*, 2019.
[31] Joshua Garcia, Yang Feng, and et al. A Comprehensive Study of Autonomous Vehicle Bugs. In *ICSE*, 2020.
[32] Jingquan Ge, Yaowen Zheng, and et al. OptRCA: A More Efficient and Accurate Approach for Automated Root Cause Analysis and Explanation. *TOSEM*, 2025.
[33] Herodotos Herodotou and Shivnath Babu. Profiling, What-If Analysis, And Cost-Based Optimization of MapReduce Programs. In *VLDB Endowment*, 2011.
[34] Zhisheng Hu, Shengjian Guo, and et al. Coverage-based Scene Fuzzing for Virtual Autonomous Driving Testing. *arXiv preprint arXiv:2106.00873*, 2021.
[35] Yuqi Huai, Yuntianyi Chen, and et al. Doppelgänger Test Generation for Revealing Bugs in Autonomous Driving Software. In *ICSE*, 2023.
[36] Seulbae Kim, Major Liu, and et al. DriveFuzz: Discovering Autonomous Driving Bugs through Driving Quality-Guided Fuzzing. In *CCS*, 2022.
[37] Taegyu Kim, Chung Hwan Kim, and et al. From Control Model to Program: Investigating Robotic Aerial Vehicle Accidents with MAYDAY. In *USENIX Security*, 2020.
[38] Laks VS Lakshmanan, Alex Russakovsky, and et al. What-if OLAP Queries with Changing Dimensions. In *ICDE*, 2008.
[39] Cheryl Lee, Tianyi Yang, and et al. Eadro: An End-to-End Troubleshooting Framework for Microservices on Multi-Source Data. In *ICSE*, 2023.
[40] Changwen Li, Chih-Hong Cheng, and et al. ComOpT: Combination and Optimization for Testing Autonomous Driving Systems. In *ICRA*, 2022.
[41] Guanpeng Li, Yiran Li, and et al. AV-FUZZER: Finding Safety Violations in Autonomous Driving Systems. In *ISSRE*, 2020.
[42] Zhongrui Li, Jiarun Dai, and et al. VioHawk: Detecting Traffic Violations of Autonomous Driving Systems through Criticality-guided Simulation Testing. In *ISSTA*, 2024.
[43] Yu Liang, Song Liu, and et al. Detecting Logical Bugs of {DBMS} with Coverage-based Guidance. In *USENIX Security*, 2022.
[44] Qingwei Lin, Hongyu Zhang, and et al. Log Clustering based Problem Identification for Online Service Systems. In *ICSE*, 2016.
[45] Guannan Lou, Yao Deng, and et al. Testing of Autonomous Driving Systems: Where Are We And Where Should We Go? In *ESEC/FSE*, 2022.

Just Accepted

[46] You Lu, Yifan Tian, and et al. DiaVio: LLM-Empowered Diagnosis of Safety Violations in ADS Simulation Testing. In *ISSTA*, 2024.
[47] Guodong Rong, Byung Hyun Shin, and et al. Lgsvl Simulator: A High Fidelity Simulator for Autonomous Driving. In *ITSC*, 2020.
[48] Diptikalyan Saha. Extending Logical Attack Graphs for Efficient Vulnerability Analysis. In *CCS*, 2008.
[49] Stefan Schmid and Jiri Srba. Polynomial-time What-if Analysis for Prefix-manipulating MPLS Networks. In *INFOCOM*, 2018.
[50] Konstantin Serebryany, Derek Bruening, and et al. AddressSanitizer: A Fast Address Sanity Checker. In *USENIX ATC*, 2012.
[51] Tong Duy Son, Ajinkya Bhave, and et al. Simulation-based Testing Framework for Autonomous Driving Development. In *ICM*, 2019.
[52] Ruoyu Song, Muslum Ozgur Ozmen, and et al. Discovering Adversarial Driving Maneuvers against Autonomous Vehicles. In *USENIX Security*, 2023.
[53] Sebastian Söntges and Matthias Althoff. Computing Possible Driving Corridors for Automated Vehicles. In *IV*, 2017.
[54] Ting Su, Yichen Yan, and et al. Fully Automated Functional Fuzzing of Android Apps for Detecting Non-crashing Logic Bugs. *PACMPL*, 2021.
[55] Huijia Sun, Christopher M. Poskitt, and et al. ACAV: A Framework for Automatic Causality Analysis in Autonomous Vehicle Accident Recordings. In *ICSE*, 2024.
[56] Jiachen Sun, Yulong Cao, and et al. Towards Robust Lidar-based Perception in Autonomous Driving: General Black-box Adversarial Sensor Attack And Countermeasures. In *USENIX Security*, 2020.
[57] Yang Sun, Christopher M Poskitt, and et al. LawBreaker: An Approach for Specifying Traffic Laws And Fuzzing Autonomous Vehicles. In *ICSE*, 2022.
[58] Haoxiang Tian, Yan Jiang, and et al. MOSAT: Finding Safety Violations of Autonomous Driving Systems Using Multi-objective Genetic Algorithm. In *ESEC/FSE*, 2022.
[59] Simon Ulbrich, Till Menzel, and et al. Defining and Substantiating the Terms Scene, Situation, and Scenario for Automated Driving. In *ITSC*, 2015.
[60] Ziwen Wan, Yuqi Huai, and et al. Towards Automated Driving Violation Cause Analysis in Scenario-Based Testing for Autonomous Driving Systems. *arXiv preprint arXiv:2401.10443*, 2024.
[61] Ziwen Wan, Junjie Shen, and et al. Too Afraid to Drive: Systematic Discovery of Semantic DoS Vulnerability in Autonomous Driving Planning under Physical-World Attacks. In *NDSS*, 2022.
[62] Jun Xu, Dongliang Mu, and et al. Postmortem Program Analysis with Hardware-Enhanced Post-Crash Artifacts. In *USENIX Security*, 2017.
[63] Chen Yan, Zhijian Xu, and et al. Rolling Colors: Adversarial Laser Exploits Against Traffic Light Recognition. In *USENIX Security*, 2022.
[64] Guangba Yu, Pengfei Chen, and et al. Nezha: Interpretable Fine-Grained Root Causes Analysis for Microservices on Multi-modal Observability Data. In *FSE*, 2023.
[65] Lingzhe Zhang, Tong Jia, and et al. Towards Close-to-Zero Runtime Collection Overhead: Raft-Based Anomaly Diagnosis on System Faults for Distributed Storage System. *IEEE Transactions on Services Computing*, 2025.
[66] Shenglin Zhang, Jun Zhu, and et al. Fault Diagnosis for Test Alarms in Microservices through Multi-source Data. In *FSE*, 2024.
[67] Xudong Zhang and Yan Cai. Building Critical Testing Scenarios for Autonomous Driving from Real Accidents. In *ISSTA*, 2023.
[68] Ziyuan Zhong, Gail Kaiser, and et al. Neural Network Guided Evolutionary Fuzzing for Finding Traffic Violations of Autonomous Vehicles. *TSE*, 2022.
[69] Rui Zhou, Ziqian Lin, and et al. Testing Scenarios Construction for Connected and Automated Vehicles Based on Dynamic Trajectory Clustering Method. In *ITSC*, 2022.
[70] Yuan Zhou, Yang Sun, and et al. Specification-based Autonomous Driving System Testing. *TSE*, 2023.

Just Accepted